VERIF-AI: A Deep Learning-Based Framework for Face-Based Deepfake Detection and Forensic Analysis

Submitted By

ANIKET RAJ

SEMESTER - VII

230103030

Under the Guidance of

DR. SUNITA WARJRI



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

NOVEMBER, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

ANIKET RAJ
230103030

DATE: _____

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Sunita Warjri
Assistant Professor

Email: sunitaw@iiitmanipur.ac.in
Contact No: +91 7085262822

To Whom It May Concern

This is to certify that the project report entitled **"VERIF-AI: A Deep Learning-Based Framework for Face-Based Deepfake Detection and Forensic Analysis,"** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfilment for the award of degree of Bachelor of Technology in Computer Science and Engineering (Specialisation: Artificial Intelligence and Data Science) is a record of bonafide work carried out by **Aniket Raj** bearing roll number **230103030**.

Signature of Supervisor

(Dr. Sunita Warjri)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

Synthetic face imagery produced by generative adversarial networks, face-swapping tools and diffusion models can now be hard to distinguish from genuine photographs, which creates a demand for forensic tools that give not only a verdict but also inspectable evidence. This report presents **Verif-AI**, an end-to-end framework for face-based deepfake detection and forensic analysis that works on both images and videos. Faces are located with MTCNN and passed to a two-stream convolutional classifier: an EfficientNet-B0 stream that reads the RGB face, and a second EfficientNet-B0 stream that reads a noise residual produced by a fixed three-kernel Spatial Rich Model (SRM) filter bank. The two feature vectors are fused by a small classification head into a single raw sigmoid score. Gradient-weighted Class Activation Mapping (Grad-CAM) shows which facial regions influenced the RGB stream's score, a video pipeline adds face tracking, frame-level timelines, suspicious time ranges and stored evidence frames, and a PDF forensic report records the result together with the model checkpoint hash and the limitations. The system is implemented as a FastAPI backend and a React/TypeScript dashboard in which uploads are deleted immediately after analysis and stored results can be deleted by the user.

The evaluation is reported as it was obtained, including its negative findings. The selected production model (v2, trained on a sample of the DeepFakeFace dataset) reaches an accuracy of 0.875 and a ROC-AUC of 0.941 on held-out identities seen-generator data, but only 0.746 and 0.849 on an unseen text-to-image generator, and it falsely flags genuine video from the Celeb-DF dataset at chance level. Diagnostic experiments traced this failure to an image-quality shortcut in the training data, and adding Celeb-DF video frames and quality augmentation (Model 4) raised the video-level accuracy on held-out Celeb-DF identities to 0.980, which is within-dataset evidence only. A further experiment with modern-generator faces (E3) detected held-out fakes of those generators almost perfectly, but without a generator-disjoint test it cannot be read as evidence of general detection, and it did not meet the pre-registered promotion rule. Scores are raw sigmoid outputs, not calibrated probabilities. The latest model, E4, is the default of the local examiner demonstration; it did not pass the promotion rule, so the public deployment still serves v2. The report does not claim a general detector of AI-generated images or videos, and the public deployment on free hosting tiers runs in a restricted mode that is not production-ready.

Keywords: deepfake detection, two-stream network, EfficientNet, SRM noise residuals, Grad-CAM, digital forensics, FastAPI, React.

Acknowledgement

I would like to express my deep gratitude to my supervisor, Dr. Sunita Warjri, Assistant Professor, Department of Computer Science and Engineering, IIIT Manipur, for her guidance and support throughout this project. Her feedback shaped both the system design and the way the results are evaluated and reported, and her encouragement made it possible to take on the more demanding parts of the work, in particular the careful treatment of evaluation leakage and of what the system can and cannot claim.

The two-stream design that Verif-AI builds on started from an earlier Forensic-Net prototype by Lakshya Singh, whose notebook, checkpoint and plots are preserved in the project repository; the web application, the training and evaluation tooling and the analysis in this report were developed on top of that starting point. I am grateful to the authors and maintainers of the open datasets and software used here (DeepFakeFace, Celeb-DF v2, FFHQ, SFHQ, PyTorch, FastAPI and React), to Kaggle for the free GPU time on which the models were trained, and to the Indian Institute of Information Technology Senapati, Manipur, for providing the environment in which this work was carried out.

Aniket Raj
Roll No: 230103030
IIIT Manipur

Contents

List of Figures vii

List of Tables vii

1 Introduction 1

1.1 Introduction . . . 1
1.1.1 Problem Statement: Verifying Faces in an Age of Synthetic Media . . . 1
1.1.2 Motivation and Project Objectives . . . 2
1.1.3 Scope and Target Applications . . . 2
1.1.4 Contributions . . . 2
1.1.5 Organisation of the Report . . . 2

2 Literature Survey 3

2.1 Literature Survey . . . 3
2.1.1 Evolution of Deepfake Generation and Detection . . . 3
2.1.2 Theoretical Framework of Modern Forensics . . . 3
2.1.3 Analysis of Benchmark Architectures . . . 4
2.1.4 Comparison of Forensic Datasets . . . 4
2.1.5 Generalisation, Shortcuts and Calibration . . . 5
2.1.6 Synthesis and Research Gap . . . 5
2.2 Research Gap, Project Contributions, and Novelty . . . 5
2.2.1 What existing research has achieved . . . 5
2.2.2 Gaps that matter for this project . . . 5
2.2.3 Established components used (not new) . . . 6
2.2.4 The prototype and what Verif-AI adds . . . 6
2.2.5 Contributions, by type . . . 7
2.2.6 What remains limited . . . 7

3 Requirement Engineering 8

3.1 Requirement Engineering and Design . . . 8
3.1.1 Project Feasibility Analysis . . . 8
3.1.2 Requirements Analysis . . . 8
3.1.3 Proposed System Architecture . . . 8
3.1.4 Hardware and Software Requirements . . . 9
3.1.5 Development Technology Stack . . . 9
3.1.6 Constraints and Assumptions . . . 9

4 System Design 13

4.1 Detailed System Design . . . 13
4.1.1 Foundational Concepts . . . 13
4.1.2 System Architecture Overview . . . 13
4.1.3 The Two-Stream Neural Architecture . . . 13
4.1.4 Image Analysis Pipeline . . . 15
4.1.5 Video Analysis Pipeline . . . 15
4.1.6 API and Data Model . . . 16
4.1.7 Storage, Privacy and Security Design . . . 16

4.1.8 User Interface and Dashboard Layout . . . 16
4.1.9 Deployment Architecture . . . 17
5 Implementation **19**
5.1 Implementation and Engineering . . . 19
5.1.1 Development Environment Setup . . . 19
5.1.2 Algorithm Specifications . . . 19
5.1.3 Module-wise Implementation Breakdown . . . 19
5.1.4 Training and Evaluation Implementation . . . 21
5.1.5 Frontend Implementation . . . 22
5.1.6 Testing and Verification . . . 24
5.1.7 Deployment Implementation . . . 25
5.1.8 Engineering Challenges and Optimisations . . . 25
6 Results and Discussion **26**
6.1 Results, Evaluation and Discussion . . . 26
6.1.1 Experimental Setup and Dataset Partitioning . . . 26
6.1.2 Performance Evaluation Methodology . . . 27
6.1.3 Results for the Production Model v2 (DeepFakeFace) . . . 27
6.1.4 A Failure on Genuine Video: the Four-Model Comparison . . . 28
6.1.5 Diagnosis: an Image-Quality Shortcut . . . 28
6.1.6 The Stage-0 Acceptance Gate . . . 29
6.1.7 Experiment E3: Modern-Generator Training . . . 29
6.1.8 Experiment E4: Source-Balanced Variant . . . 31
6.1.9 Qualitative Analysis of Grad-CAM Output . . . 32
6.1.10 Verification of the Deployed System . . . 32
6.1.11 Limitations and Threats to Validity . . . 33
6.1.12 Summary of Results . . . 33
7 Conclusion and Future Scope **34**
7.1 Conclusion and Future Perspectives . . . 34
7.1.1 Project Summary and Key Contributions . . . 34
7.1.2 Final Performance Assessment . . . 34
7.1.3 Critical Reflections and Limitations . . . 34
7.1.4 Future Perspectives . . . 35
7.1.5 Closing Remarks . . . 35
A Source Code Snippets **36**
A.1 The Custom SRM Filter Layer and the Two-Stream Network . . . 36
A.2 Preprocessing Shared by Training and Inference . . . 36
A.3 Scoring Rules . . . 37
A.4 Face Detection . . . 37
A.5 Grad-CAM . . . 38
B API and Configuration Reference **38**
C Dataset and Licence Review **39**
D Reproducibility **39**
Bibliography **40**

List of Figures

1.1 Conceptual overview of Verif-AI 1

2.1 Illustration (schematic, not a real face) of the idea behind the two streams: the RGB stream . 4

3.1 Data-flow diagram of Verif-AI. Processes 1–6 run in the API process; D1 is the results store (evidence images and JSON), which is deleted on request or after the retention period 11
3.2 Use-case diagram. The reviewer interacts with six use cases; the operator configures and . . 11

4.1 System architecture of Verif-AI 14
4.2 Two-stream architecture of Verif-AI: 11,972,954 parameters in total (two EfficientNet-B0 . . 14
4.3 Image pipeline. Faces below 96 pixels are analysed but flagged as low resolution; an image . 15
4.4 Video pipeline. The aggregation (median per track, maximum over tracks) was chosen a priori 16
4.5 Sequence of a video analysis. The browser polls the job status; the upload is deleted by the . 16
4.6 Home page of the dashboard (current dark interface; screenshot taken from the running . . 17
4.7 Deployment architecture of the public demonstration 18

5.1 Pages 1 and 2 of a PDF forensic report generated by the application for a public test image . 22
5.2 Heat-map viewer for an image result (public test image) 23
5.3 Frame Analysis tab for a video (a synthetic clip made from the public test image): the 23
5.4 Honest unavailability and face tracks 24
5.5 Overview tab of an image result: verdict, model score labelled as uncalibrated, analysis summary, the heat-map viewer and the detected faces with bounding boxes 24

6.1 ROC curves of v2 on the two held-out DeepFakeFace test splits (generated by the evaluation 27
6.2 Confusion matrices of v2 at threshold 0.5 28
6.3 Fake share per fixed sharpness bin in DeepFakeFace (values from the E1 diagnostic) 29

List of Tables

2.1 Backbones used in the deepfake-detection literature (published figures). 4
2.2 Datasets referenced in this work (figures as stated by the dataset papers or by the dataset . . 4
2.3 Existing work and how Verif-AI relates to it (only capabilities stated in the cited papers or . 6

3.1 Functional requirements (FR) of Verif-AI. 9
3.2 Non-functional requirements (NFR) and how each was addressed or measured. 10
3.3 Main use cases. 11
3.4 Hardware and software requirements. 12
3.5 System technology stack and development environment. 12

4.1 REST endpoints (prefix `/api`). 17

5.1 Core modules and file structure of the repository. 20
5.2 Backend memory (process working set, MB) measured on the development laptop for different 25

6.1 DeepFakeFace splits used for the production model (face crops). 26
6.2 Composition of the experiment E3 manifest after face detection and near-duplicate removal . 26
6.3 Production model v2 on DeepFakeFace (threshold 0.5; fake is positive; intervals are 95% . . 27
6.4 Video-level results on 147 held-out Celeb-DF videos (91 real, 56 fake), clean inputs, 95% . . . 28
6.5 Stage-0 gate results (validation data, clean and degraded sets; DFF = DeepFakeFace, CDF = 29
6.6 Frame-level results, clean inputs, fixed threshold 0.5 (FPR: real called FAKE; FNR: fake called REAL; 95% Wilson intervals) 30
6.7 Video-level results on the Celeb-DF held-out identities (91 real and 56 fake videos), clean . . 30
6.8 Share of crops called FAKE per source, clean inputs (for real sources this is the 30
6.9 Robustness to a class-blind *mixed* degradation (blur, down-up resampling, JPEG, noise . . . 31
6.10 The frozen promotion rule applied to E3A (rule written before training). 31
6.11 Experiment E4 against Model 4 and v2: held-out results 32
6.12 The frozen promotion rule applied to E4 (rule written before training). 32

B.1 Backend environment variables. 38

C.1 Decisions of the dataset review. 39

Chapter 1

Introduction

1.1 Introduction

Photographs and video of a person's face have long served as informal evidence of identity and events. Generative models have weakened that assumption: generative adversarial networks (GANs) [1, 2], face-swapping and re-enactment tools [3] and, more recently, diffusion models [4, 5] can produce or alter faces whose artefacts are invisible to the naked eye. Research on *media forensics* [6, 7] responds with automatic detectors, but a detector that only prints "fake" or "real" is of limited use to a reviewer: the verdict has to be accompanied by evidence that can be inspected, by the conditions under which it is reliable, and by an honest statement of what it cannot decide.

Verif-AI is a software system built around that idea. It accepts an image or a video, finds the faces in it, scores every face with a two-stream convolutional network, shows which facial regions influenced the score, summarises a video over time and generates a PDF forensic report. This report describes the design, the implementation, and — with equal weight — the evaluation of the system, including the experiments that showed where it fails.

1.1.1 Problem Statement: Verifying Faces in an Age of Synthetic Media

The practical problem has three parts.

1. **Detection.** Given a face crop, estimate whether it shows a photographed face or a generated or manipulated one. Detectors trained on one family of manipulations often transfer poorly to another [8–10], so the measured accuracy depends heavily on how the test data relate to the training data.
2. **Evidence.** A score alone does not let a reviewer judge a result. The system must localise what drove the decision, report the detection confidence of the face finder, show where in a video suspicious frames occur and keep a record (file hash, model checkpoint, settings) that makes the analysis reproducible.
3. **Honesty and privacy.** Neural scores are not probabilities of authenticity [11], detectors are exposed to *shortcuts* in their training data [12], and uploaded media can be sensitive. The system must therefore say "inconclusive" when it cannot decide, avoid presenting scores as certainties, and delete uploads after use.

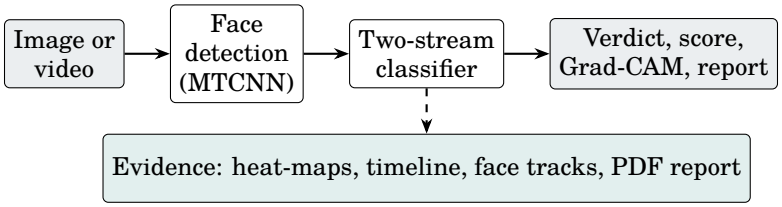


Figure 1.1: Conceptual overview of Verif-AI. A face is located, scored by a two-stream network and returned together with inspectable evidence. Scores are raw model outputs (not probabilities); INCONCLUSIVE is an explicit outcome.

1.1.2 Motivation and Project Objectives

The project started from an earlier prototype that combined an RGB and an SRM noise stream and reported very high accuracy on a single dataset. When the prototype's checkpoint was evaluated on independent, identity-disjoint data it performed at chance level, which made the central motivation of this work concrete: *a detector is only as trustworthy as its evaluation*. The objectives were therefore both engineering and scientific:

- **Objective 1:** to re-implement the two-stream detector as a clean, tested software package that loads the original checkpoints and a newly trained one, with identical preprocessing for training and inference.
- **Objective 2:** to train and evaluate the model with identity-disjoint splits, duplicate removal, validation-only model selection and a clear separation of internal and cross-generator evaluation.
- **Objective 3:** to provide explainable output — Grad-CAM heat-maps, face-level scores, video timelines, face tracks and a PDF report — through an API and a web dashboard, with privacy-preserving handling of uploads.
- **Objective 4:** to investigate honestly where the model fails (for example on genuine video) and to document the cause and the limits of every claim.
- **Objective 5:** to deploy a public demonstration (static frontend, hosted API) and to state its restrictions truthfully.

1.1.3 Scope and Target Applications

Verif-AI analyses *faces*. It is intended for educational demonstration, exploratory review of suspicious face media by a human and research on detector generalisation. It is **not** a general detector of AI-generated images or videos, does not analyse non-face content, does not establish authorship or provenance, and a "real" result is not proof of authenticity. Scores are uncalibrated; these boundaries decide how every number in Chapter 6 may be read.

1.1.4 Contributions

1. A tested two-stream detection package with a documented preprocessing pipeline that reproduces training-time cropping, JPEG round-trip and normalisation at inference time (Chapters 4 and 5).
2. An image and video analysis service with Grad-CAM evidence, face tracking, frame timelines, PDF reports and deletion (Chapter 5), and a dashboard with original, heat-map and overlay views.
3. An evaluation record that includes an identified failure (chance-level behaviour on genuine video), a diagnosis of its cause as an image-quality shortcut, a pre-registered acceptance gate and modern-generator experiments (Chapter 6).
4. A reproducible training and data-review workflow (Kaggle GPU notebooks that export only checkpoints and metrics) and a dataset licence review that did not accept licence badges as proof (Chapter 6 and Appendix C).

1.1.5 Organisation of the Report

Chapter 2 surveys the field, the research gap and the contributions of this work; Chapter 3 the requirements and technology stack; Chapter 4 the architecture, model and pipelines; Chapter 5 their implementation, testing and deployment; Chapter 6 the experiments, their interpretation and limitations; Chapter 7 the conclusion and future work. Appendices A–C give key source code, the API and configuration, and the dataset review.

Chapter 2

Literature Survey

2.1 Literature Survey

This chapter summarises the research that Verif-AI builds on and the open problems that shaped its evaluation. It is not a complete survey; broader reviews are given by Verdoliva [6] and Tolosana et al. [7].

2.1.1 Evolution of Deepfake Generation and Detection

Generation. Generative adversarial networks [1] made photorealistic face synthesis practical; StyleGAN and its successors generate high-resolution faces from a latent code [2, 13]. Face-swapping and re-enactment methods transfer one person's face or expression onto another's video, and the FaceForensics++ benchmark collects four such manipulation families [3]. Diffusion models [4], in particular latent diffusion [5], now produce images from text prompts and can inpaint or edit regions of real photographs, so a single dataset may contain fully synthetic, edited and swapped faces.

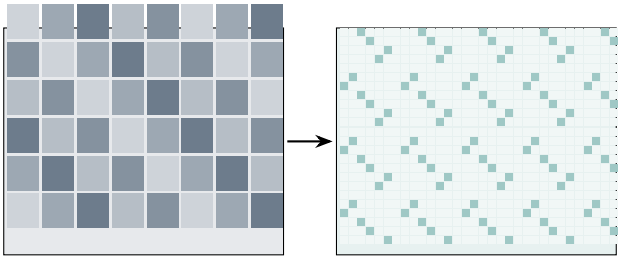
Detection. Early detectors used compact convolutional networks on face crops, such as MesoNet [14], or general-purpose backbones such as Xception [15] that performed best in the FaceForensics++ study [3]. Two-stream networks combine an RGB stream with a stream that sees low-level traces: Zhou et al. [16] paired a face-classification stream with a steganalysis-features stream for tampered faces, and constrained convolutional layers [17] learn high-pass filters for manipulation detection. Dang et al. [18] add attention maps that localise manipulated regions. Large benchmark efforts such as Celeb-DF [19] and the DFDC collection [20] made video-level evaluation possible.

2.1.2 Theoretical Framework of Modern Forensics

RGB versus noise residuals. A generator leaves statistical traces in the high-frequency content of an image that are largely independent of the depicted face. Rich-model steganalysis [21] extracts such traces by applying fixed high-pass *residual* filters and analysing the co-occurrence of the residuals. Verif-AI uses a fixed bank of three 3×3 high-pass kernels (Chapter 4) that suppress image content and leave a noise residual for a second convolutional stream. The filters are not trained, which keeps the stream's input independent of the training data.

EfficientNet. EfficientNet [22] scales depth, width and resolution of a convolutional network with a compound coefficient. The smallest member, EfficientNet-B0, has about 5.3 million parameters and reports 77.1% ImageNet top-1 accuracy, which makes two parallel copies affordable on a CPU.

Grad-CAM. Gradient-weighted Class Activation Mapping [23] weights the activation maps of a chosen convolutional layer by the average gradient of the class score with respect to them and shows the result as a coarse heat-map. The map indicates regions that increased the score; it is an explanation of the *model*, not evidence that those pixels were altered [23]. Figure 2.1 contrasts the information seen by the two streams.



RGB face (what a person sees)

High-pass residual (what the SRM stream sees)

Figure 2.1: Illustration (schematic, not a real face) of the idea behind the two streams: the RGB stream sees image content, whereas the fixed SRM-style filters remove content and leave a high-frequency noise residual in which generator traces may remain.

2.1.3 Analysis of Benchmark Architectures

Table 2.1 compares backbones commonly used for face-manipulation detection. The figures are the sizes and ImageNet results reported by the original papers; they describe the architectures, not their forensic accuracy, which depends on training data.

Table 2.1: Backbones used in the deepfake-detection literature (published figures).

Architecture	**Params (M)**	**ImageNet top-1**	**Relevance**
MesoNet (Meso-4) [14]	0.028	–	Very small network designed for face-video forgery
Xception [15]	22.9	79.0%	Strongest baseline in FaceForensics++ [3]
ResNet-50 [24]	25.6	–	Common general-purpose baseline
EfficientNet-B0 [22]	**5.3**	**77.1%**	Used for both streams of Verif-AI

2.1.4 Comparison of Forensic Datasets

Table 2.2 lists the datasets that matter for this project. Their *construction* matters as much as their size: how real and fake samples are paired, whether identities repeat across splits, and how fakes were compressed or processed all create shortcuts that a detector can learn instead of the manipulation itself (Section 2.1.5).

Table 2.2: Datasets referenced in this work (figures as stated by the dataset papers or by the dataset documentation in the repository).

Dataset	**Content**	**Fakes**	**Use in Verif-AI**
FaceForensics++ [3]	1,000 original videos	4 manipulation methods	Cited; not used (access agreement)
DFDC [20]	over 100,000 clips	Several methods	Cited; not used
Celeb-DF v2 [19]	590 real and 5,639 fake videos	Face swap (one method)	Video-level training and test (research use only)
DeepFakeFace [25]	30,000 real (IMDB-WIKI) photographs [26]	3×30,000: inpainting, face swap, text-to-image	Primary training data (sampled subset)
FFHQ [2]	70,000 real faces	–	Real faces in experiment E3
SFHQ [27]	Synthetic faces	StyleGAN-generated	Fake faces in experiment E3

2.1.5 Generalisation, Shortcuts and Calibration

Generalisation. A classifier trained on images from one generator can be nearly perfect on that generator and weak on others. Wang et al. [8] found that a classifier trained on one GAN with suitable augmentation generalised surprisingly well to other CNN-based generators, while stressing the limits of that result; Ojha et al. [9] and Corvi et al. [10] study the transfer between GAN and diffusion generators, where detectors trained on one family often degrade on the other. A reliable evaluation therefore needs a held-out *generator*, not only held-out images.

Shortcut learning. Geirhos et al. [12] describe how networks solve benchmarks with unintended cues. In deepfake data the cue may be image sharpness, resolution, file format, compression history or the source of the real class. Chapter 6 documents a concrete case in this project.

Calibration and explanation. The sigmoid output of a modern network is usually over-confident [11]; Verif-AI therefore labels it a *model score* and never a probability. Grad-CAM maps are explanations of the model's decision, and as noted above they do not prove manipulation.

2.1.6 Synthesis and Research Gap

Prior work supplies the ingredients — an efficient backbone, a noise-residual stream, class-activation explanations and benchmark datasets — but a practical forensic tool also needs a consistent preprocessing pipeline, video aggregation, privacy-preserving handling of uploads and an evaluation that distinguishes internal from cross-generator behaviour. The gap addressed in this report is therefore less a new architecture than a *complete and honestly evaluated system*: the two-stream design is kept, while the data splits, leakage checks, failure diagnosis and the limits of each claim are treated as first-class results.

2.2 Research Gap, Project Contributions, and Novelty

This section states what existing work has achieved, which gaps matter for this project, which components are borrowed, and what Verif-AI itself contributes. **Verif-AI does not propose a new deep-learning architecture**, and it does not claim to have invented EfficientNet, SRM filtering, MTCNN or Grad-CAM; its contributions are in engineering, system integration and experimental method.

2.2.1 What existing research has achieved

Benchmarks and generators: FaceForensics++ [3], Celeb-DF [19], the DFDC dataset [20] and DeepFakeFace [25] supply labelled data. Detectors: compact and Xception-style CNNs [3, 14]; two-stream networks that combine an RGB branch with a low-level residual branch [16]; detectors that exploit high-frequency/SRM-filtered features to generalise across forgeries [28]; blending-boundary cues for unseen manipulations [29]; recurrent video models over frame features [30, 31]; and detectors studied across generators [8–10]. Explanation: Grad-CAM [23]. Evaluation caveats: shortcut learning [12] and over-confident scores [11]. Surveys: [6, 7].

2.2.2 Gaps that matter for this project

G1. The cited papers present *methods and benchmark results*; they are not described as ready-to-use tools that return per-face evidence, video timelines and a record of which model and file produced a result.

G2. Reported accuracy depends on the split. Identity-disjoint, duplicate-free and generator-disjoint testing, and the check that an apparent skill is not a quality shortcut, must be done again for any new system.
G3. Video support is usually a temporal classifier validated inside one dataset; a deployed tool also needs face tracking, aggregation and honest handling of frames for which no evidence was stored.
G4. A prototype inherited by this project (Forensic-Net, Section 2.2.4) reported very high accuracy that could not be reproduced on independent data, so the model itself needed to be re-trained and re-evaluated.

2.2.3 Established components used (not new)

EfficientNet-B0 [22] as both backbones; the three SRM-style high-pass kernels after Fridrich and Kodovský [21]; the two-stream idea of an image branch plus a low-level residual branch [16]; MTCNN [32]; Grad-CAM [23]; ImageNet initialisation, AdamW [33] and PyTorch [34]; and the public datasets of Chapter 6.

2.2.4 The prototype and what Verif-AI adds

The RGB + three-kernel SRM two-stream design, with its feature concatenation and sigmoid head, comes from the earlier Forensic-Net prototype written by Lakshya Singh (notebook, checkpoint and plots preserved in the repository). That prototype analysed only the first frame of a file and had no application, no leakage-controlled evaluation and no tests. Everything listed below was developed for Verif-AI.

Table 2.3: Existing work and how Verif-AI relates to it (only capabilities stated in the cited papers or in this repository).

Existing work	**What it contributes**	**Verif-AI: implemented / extended / different**
Zhou et al. [16]	two-stream face-tampering detector (image + low-level branch)	uses the two-stream idea with fixed SRM residuals; adds identity-disjoint training/evaluation and a per-quality-bin gate
Fridrich & Kodovský [21]	rich residual models for steganalysis	three fixed kernels as the noise stream; examines when that stream carries a quality shortcut (Section 6.1.5)
Luo et al. [28]; Li et al. [29]	generalisation through high-frequency features / blending boundaries	not re-implemented; the project measures generalisation with an unseen-generator split instead
Güera & Delp [30]; Masi et al. [31]	recurrent networks over frame features for video	different approach: per-face scores, IoU face tracks, per-track median, suspicious segments and stored evidence frames (unvalidated aggregation rule)
Selvaraju et al. [23]	Grad-CAM	applied to the RGB stream, shown with Original/Heatmap/Overlay views, stored for evidence frames, stated as not proof
Wang et al., Ojha et al., Corvi et al. [8–10]	detectors studied across generators	motivates the internal vs. cross-generator reporting; no claim of universal detection
Celeb-DF, DeepFake-Face, FFHQ/SFHQ [2, 19, 25, 27]	datasets	used under their licences; leakage control, near-duplicate removal and a licence review before use

2.2.5 Contributions, by type

Research novelty. None claimed at the level of a new architecture or algorithm. The research-level results are empirical findings: (i) the production model was at chance on held-out Celeb-DF video; (ii) image sharpness is a measurable shortcut in DeepFakeFace and the noise stream carries it; (iii) a per-quality-bin acceptance gate (validated with synthetic controls) fails all four candidate models although two pass the usual ROC-AUC screen.

Engineering novelty. Training-matched preprocessing (JPEG round-trip), strict checkpoint loading with hash recording, an INCONCLUSIVE band and decisiveness value, atomic job storage, privacy-preserving deletion, memory-aware lite mode, and a model registry (default / alternative / rollback) with per-result model identification.

System-level integration. One React + FastAPI application that turns the two-stream model into an inspectable workflow: face detection, image and video analysis, face tracking, Grad-CAM, timelines, evidence frames, downloadable PDF forensic report, history and deletion, deployed as a static frontend plus a hosted API.

Experimental contribution. A reproducible Kaggle training and evaluation workflow with pre-registered protocols; four models compared under identical code (Table 6.4); experiments E1–E4 including a frozen promotion rule; E4 as the latest model, with its limits reported (Section 6.1.8).

2.2.6 What remains limited

No new detector architecture, no validation of the video aggregation rule on labelled videos, no calibration, no claim about AI-generated video, E4 not approved by the promotion rule, and the live deployment serves v2 (Chapter 6, Section 6.1.11).

Chapter 3

Requirement Engineering

3.1 Requirement Engineering and Design

The transition from a research prototype to a usable forensic tool requires explicit requirements. They were derived from three sources: the weaknesses found when the earlier prototype was evaluated, the needs of a reviewer who must trust and audit a result, and the constraints of free hardware and open datasets. This chapter states them and the resulting technology choices.

3.1.1 Project Feasibility Analysis

Technical feasibility. Two EfficientNet-B0 networks (about 12 million parameters together) run on a CPU: a single image is analysed in roughly 0.5–2.5 seconds on the development laptop, and a video of a few seconds in several seconds (measured values from the analysis records). Training needs a GPU, which was obtained from the free Kaggle Tesla T4 allowance; one training run takes between about one and several hours depending on the amount of data.

Operational feasibility. The system is delivered as a Docker image or as two parts (static React build and a Python API), uses no external AI service or API key, and needs no account for the end user.

Economic feasibility. All software is open source, and training and hosting used free tiers. The free hosting tier limits what the public demonstration can do (Section 5.1.7); a plan with more memory would be required for the full feature set.

Legal and ethical feasibility. Face data are personal data. The system deletes uploads right after analysis, avoids a public list of other users' analyses, and was developed with datasets whose terms were reviewed (Appendix C). Datasets with research-only terms are used for non-commercial study only.

Main risks. (i) Over-optimistic evaluation because of leakage or shortcuts; (ii) a detector that fails on genuine video or on new generators; (iii) over-interpretation of scores or heat-maps by users. They are addressed by the evaluation design of Chapter 6 and by the wording of the user interface.

3.1.2 Requirements Analysis

Stakeholders. (1) A *reviewer* who uploads media and needs a verdict with inspectable evidence; (2) a *student or researcher* who wants a reproducible detector for experiments; (3) an *operator* who deploys and maintains the service.

3.1.3 Proposed System Architecture

The system is organised in three tiers (Figure 3.1): a browser dashboard, an HTTP API, and the analysis core. The data-flow diagram follows a single request from upload to result. Detailed designs of the model and the pipelines are in Chapter 4.

Table 3.1: Functional requirements (FR) of Verif-AI.

ID	Requirement	Status
FR-1	Accept an image or a video through one upload control; detect the media type automatically; reject unsupported or oversized files with a clear message.	Implemented
FR-2	Detect faces with a published detector and analyse every sufficiently large and confident face (up to 10 per image).	Implemented
FR-3	Score each face with the two-stream model and return a raw fake score in $[0, 1]$ together with a REAL / FAKE / INCONCLUSIVE label.	Implemented
FR-4	Report "no usable face" as INCONCLUSIVE rather than as real or fake.	Implemented
FR-5	Provide Grad-CAM heat-maps for faces and for evidence frames; show original, heat-map and overlay views.	Implemented (when the host allows)
FR-6	For video: sample frames, track faces, aggregate per track, list suspicious time ranges and store the highest-scoring frames as evidence.	Implemented
FR-7	Show the frame-level score timeline and let the user select a frame and see its stored image and heat-map.	Implemented
FR-8	Generate a PDF forensic report with the verdict, scores, file hash, model checkpoint hash, evidence and limitations.	Implemented
FR-9	Run video analysis as a background job with progress reporting.	Implemented
FR-10	Let the user view earlier analyses and delete a result at any time; delete uploads immediately after analysis; expire results automatically.	Implemented
FR-11	Expose model information (name, architecture, checkpoint hash, thresholds, limits) through the API and the interface.	Implemented
FR-12	Provide documented REST endpoints with typed error responses and a health check.	Implemented
FR-13	Detect modern text-to-image and diffusion-edited faces and AI-generated video reliably.	**Not achieved** (see Chapter 6)

3.1.4 Hardware and Software Requirements

3.1.5 Development Technology Stack

3.1.6 Constraints and Assumptions

The detector sees a single cropped face at 224×224 pixels; it has no information about audio, context or provenance. Training data are limited to the datasets listed in Table 2.2, whose licences permit non-commercial research. Free hosting tiers have no persistent disk and limited memory, so the public demonstration is reduced in capability. Video evaluation was possible only on Celeb-DF, which contains one face-swap method.

Table 3.2: Non-functional requirements (NFR) and how each was addressed or measured.

Category	**Requirement**	**Measure / mechanism**
Performance	An image is analysed within a few seconds on a CPU; video does not block the browser.	Measured 0.5–2.5 s per image on the development laptop; video runs as a polled background job.
Resource use	Run on a modest machine.	Measured on the laptop: about 490 MB after the model loads, about 780 MB peak for one image with Grad-CAM, over 820 MB while analysing a video.
Privacy	Do not retain uploads or expose other users' analyses.	Uploads deleted right after analysis; results expire (72 h default); the global history list can be disabled; results are reachable only by a 128-bit random identifier.
Security	Resist malformed and malicious uploads.	Extension allow-list, size and pixel limits, decoding check, server-chosen file names, CORS restricted to configured origins.
Honesty	Never present a score as a probability or a heat-map as proof.	Fixed wording in the interface and report; INCONCLUSIVE band; limitation text on every result.
Reproducibility	The result must be traceable to a model and an input.	File SHA-256 and checkpoint SHA-256 stored in every result and report.
Portability	Run on Windows, Linux and in a container.	Docker image; CPU and CUDA paths; environment-variable configuration.
Maintainability	Changes must be verifiable.	Automated tests: 152 backend tests passed (7 environment-gated skips), 63 frontend unit tests and 9 browser end-to-end tests at the time of writing.
Usability	Work on desktop, tablet and phone.	Responsive layout verified at 1440, 1024, 768, 390 and 320 px widths.

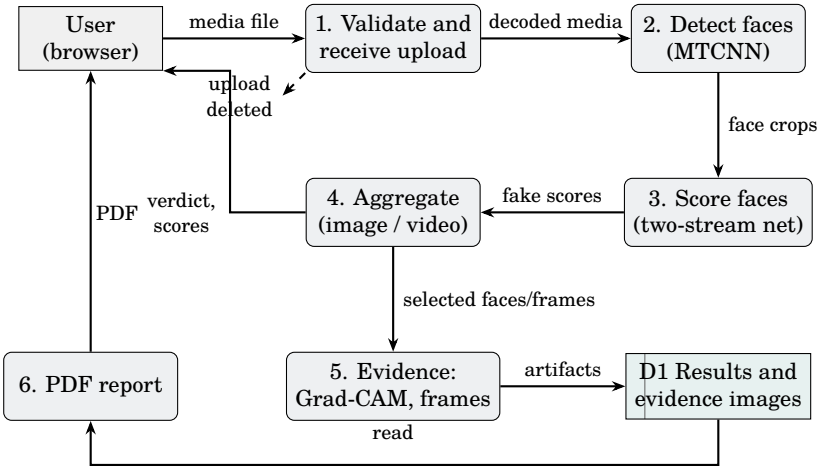


Figure 3.1: Data-flow diagram of Verif-AI. Processes 1–6 run in the API process; D1 is the results store (evidence images and JSON), which is deleted on request or after the retention period. The original upload is deleted as soon as processing 4 finishes.

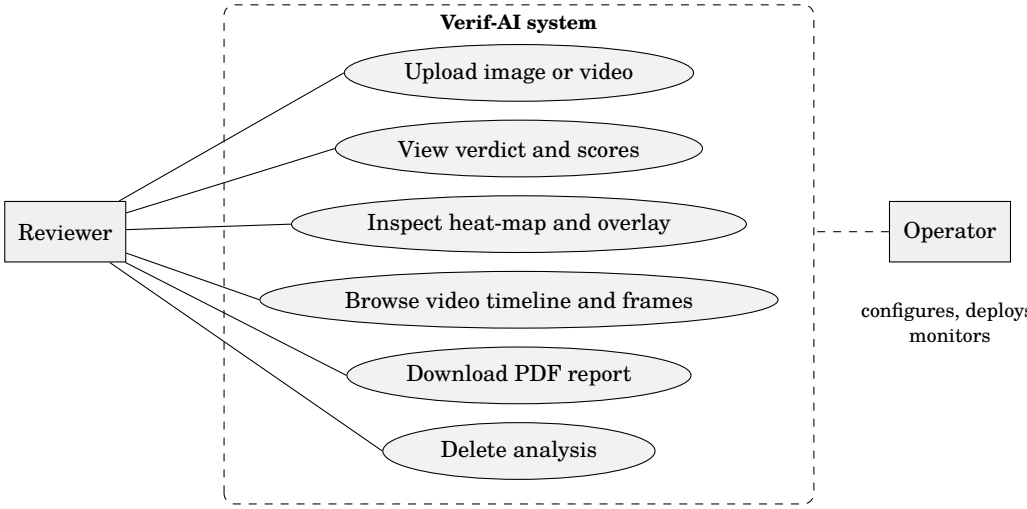


Figure 3.2: Use-case diagram. The reviewer interacts with six use cases; the operator configures and deploys the service (environment variables, hosting) but does not see uploaded content.

Table 3.3: Main use cases.

Use case	**Flow**
Analyse an image	The user selects an image; the dashboard previews it and sends it to the image endpoint; the API validates and decodes it, detects faces, scores them, generates heat-maps, deletes the upload and returns the result; the dashboard opens the result page.
Analyse a video	The user selects a video; the API stores it temporarily, starts a background job and answers immediately with an identifier; the dashboard polls the job status and opens the result when it is complete; frames, tracks and evidence are stored.
Inspect evidence	On the result page the user switches between original, heat-map and overlay views, selects a face, or selects a frame on the timeline; where no heat-map was stored the interface states this explicitly.
Delete	The user deletes the analysis; the results folder is removed and the history entry disappears.

Table 3.4: Hardware and software requirements.

Item	**Requirement**
Runtime CPU / RAM	Any modern x86-64 CPU; at least 2 GB RAM recommended (about 0.8 GB peak was measured for one analysis with Grad-CAM).
GPU	Optional for inference (CUDA is used automatically); required only for training (Kaggle Tesla T4 used).
Storage	About 50 MB per checkpoint; evidence images of an analysis are small JPEG files.
Operating system	Windows 11 (development), Linux (Docker image based on Python 3.10, Kaggle notebooks on Python 3.13).
Backend	Python 3, PyTorch, FastAPI, OpenCV, Pillow, ReportLab (see Table 3.5).
Frontend	A current Chromium-, Firefox- or Safari-class browser; Node.js 22 to build.

Table 3.5: System technology stack and development environment.

Category	**Tool / library**	**Purpose**
Deep learning	PyTorch [34]	Tensors, autograd, model definition and training
Backbone	efficientnet-pytorch 0.7.1	EfficientNet-B0 feature extractor [22]
Face detection	facenet-pytorch (MTCNN)	Face boxes and confidences [32]
Explainability	grad-cam package	Grad-CAM heat-maps [23]
Image / video	OpenCV (headless), Pillow	Decoding, resizing, JPEG round-trip, sampling
API	FastAPI, Uvicorn, Pydantic v2	REST endpoints, validation, background jobs
Reports	ReportLab	PDF forensic report
Frontend	React 18, TypeScript 5, Vite 5, Tailwind CSS 3	Dashboard and result viewers
Testing	pytest, Vitest, Testing Library, Playwright	Unit, component and browser end-to-end tests
Training	Kaggle notebooks (Tesla T4), AdamW [33]	Model training and evaluation
Delivery	Docker, Vercel, Render	Container image, static frontend hosting, API hosting

Chapter 4

System Design

4.1 Detailed System Design

This chapter gives a granular description of the Verif-AI architecture: the tiers of the system, the two-stream model, the image and video pipelines, the API and storage design, the user interface and the deployment architecture. Implementation details follow in Chapter 5.

4.1.1 Foundational Concepts

Face-level analysis. The detector never sees a whole image or frame; it sees a 224×224 crop of one face. An image with several faces is scored face by face, and the image score is the maximum over faces. A frame with no usable face yields no score.

Raw score, label and abstention. The network outputs $P_{\text{fake}} \in [0, 1]$ through a sigmoid. The decision threshold is fixed at 0.5 and is never changed after training; a separate *reporting* rule labels a result INCONCLUSIVE when $|P_{\text{fake}} - 0.5| < 0.15$. The band abstains instead of forcing a verdict and does not alter the threshold. A "decisiveness" value $2|P_{\text{fake}} - 0.5|$ is displayed with the explicit note that it is not a probability of correctness.

4.1.2 System Architecture Overview

Figure 4.1 shows the three tiers. The browser dashboard never contains model logic. The API validates input, runs the analysis in a worker, and serves results and evidence images. The analysis core is a Python package with no web dependency, which is why the same code is used by the API, the training and evaluation tools and the tests.

4.1.3 The Two-Stream Neural Architecture

The classifier is a late-fusion network (Figure 4.2). A normalised face tensor $x \in \mathbb{R}^{3\times224\times224}$ is processed by two parallel branches whose features are concatenated.

4.1.3.1 Stream 1: The Spatial Feature Extractor (RGB Stream)

The first stream is an EfficientNet-B0 [22] without its classification layer. Its last feature map has 1280 channels on a 7×7 grid and is reduced by global average pooling to a 1280-dimensional vector. This stream reads image content: blending boundaries, lighting inconsistencies, texture statistics and structural distortions. Training used ImageNet initialisation for the selected production model.

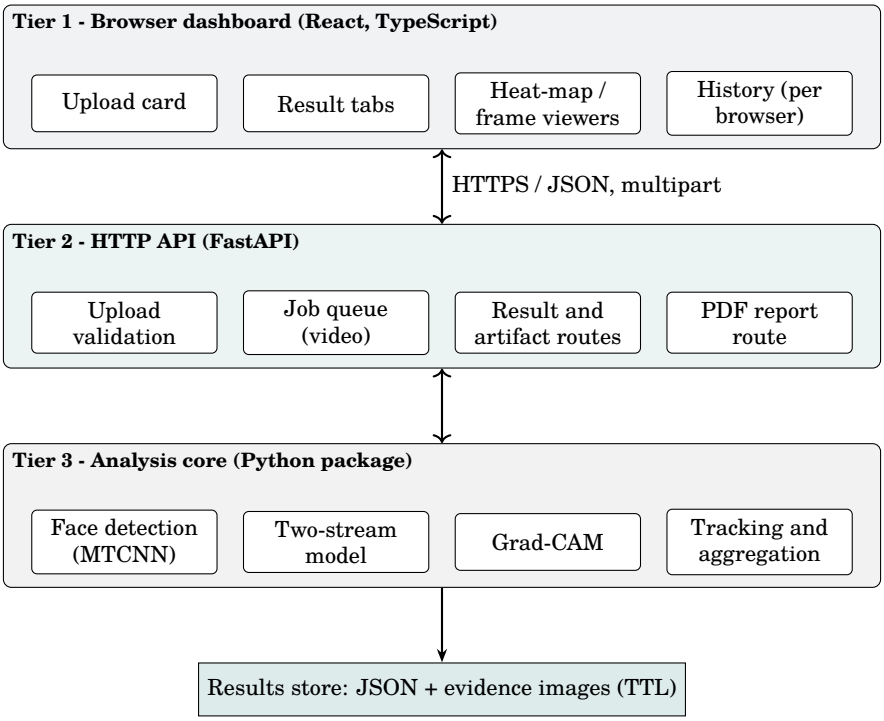


Figure 4.1: System architecture of Verif-AI. The core package is independent of the web layer and is shared by the API, the training and evaluation tools and the tests.

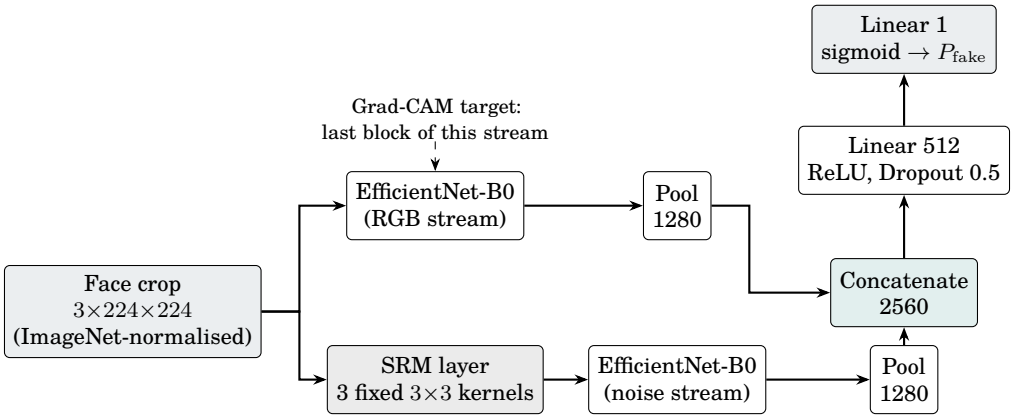


Figure 4.2: Two-stream architecture of Verif-AI: 11,972,954 parameters in total (two EfficientNet-B0 feature extractors, a fixed SRM filter bank with no trainable weights, and a 2560→512→1 head).

4.1.3.2 Stream 2: The Forensic Noise Stream (SRM Filter Layer)

The second stream first passes the same input through a fixed 3×3 convolution with three output channels. Each output channel k is $R_k = \sum_{c \in \{R,G,B\}} K_k * x_c$ with

$$K_1 = \tfrac{1}{4}\begin{pmatrix} -1 & 2 & -1 \\ 2 & -4 & 2 \\ -1 & 2 & -1 \end{pmatrix}, \quad K_2 = \tfrac{1}{4}\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}, \quad K_3 = \tfrac{1}{4}\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}. \tag{4.1}$$

These are high-pass kernels in the spirit of the rich-model residuals of Fridrich and Kodovský [21] (the original rich model uses many more filters). The kernels are constants stored with `requires_grad=False`. The resulting three-channel residual is fed to a second EfficientNet-B0 and pooled to 1280 dimensions. Whether this stream carries

manipulation evidence or merely image-quality information is examined experimentally in Chapter 6.

4.1.3.3 Feature Fusion and Classification Head

The two vectors are concatenated into a 2560-dimensional feature, passed through Linear(2560→512), ReLU, Dropout(0.5) and Linear(512→1) and a sigmoid. The head is trained jointly with both backbones. The network cannot run one stream alone; when stream-level behaviour is analysed (Chapter 6) the other stream's features are replaced by a fixed reference mean and the resulting scores are reported as *conditional* scores.

4.1.3.4 Explainability Module: Grad-CAM Integration

Grad-CAM is computed on the last convolutional block of the *RGB* stream with the single output as target. The heat-map is resized to the 224×224 crop and blended onto it; the stored artifact is this blended image. The report and the interface state that the map shows regions that influenced the RGB stream's score and does not explain the noise stream or prove manipulation.

4.1.4 Image Analysis Pipeline

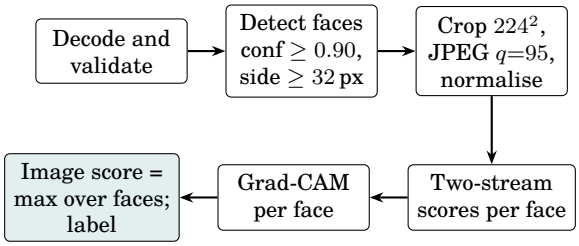


Figure 4.3: Image pipeline. Faces below 96 pixels are analysed but flagged as low resolution; an image with no usable face is reported as INCONCLUSIVE.

Preprocessing at inference reproduces training exactly: the crop is encoded as JPEG at quality 95 and decoded again, because the training crops were stored that way, and is then normalised with the ImageNet mean and standard deviation. The detection runs on a downscaled copy (longest side at most 1280 pixels) for speed while the crop itself is taken from the original image.

4.1.5 Video Analysis Pipeline

A video is analysed as a background job so that long files do not block a request (Figure 4.4).

Faces in successive frames are linked into tracks by bounding-box overlap; tracks supported by fewer than three analysed frames are ignored. The score of a track is the median of its frame scores, which reduces the effect of a few outlying frames, and the video score is the maximum over tracks. Temporal statistics (score standard deviation, mean absolute change, longest run above 0.5 and the number of boundary crossings) are reported to help a reviewer judge stability. For each of the six highest-scoring frames the system stores the frame with face boxes and a Grad-CAM heat-map of the highest-scoring face; other frames have no stored image, and the interface says so instead of approximating one.

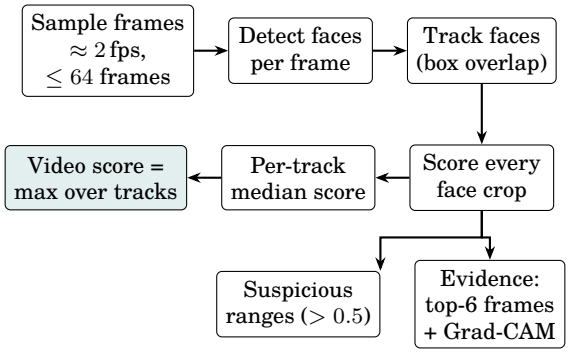


Figure 4.4: Video pipeline. The aggregation (median per track, maximum over tracks) was chosen a priori and has *not* been validated on labelled videos; the interface says so.

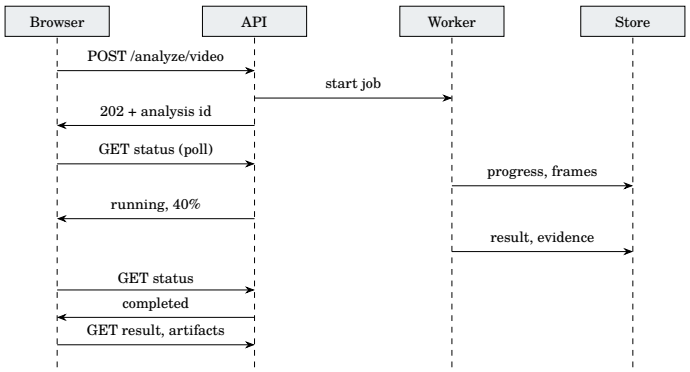


Figure 4.5: Sequence of a video analysis. The browser polls the job status; the upload is deleted by the worker when processing ends.

4.1.6 API and Data Model

A result contains the verdict, raw scores, per-face boxes, detector confidences and artifact names, for videos the per-frame records, tracks, suspicious segments, evidence frames and temporal statistics, the model description (name, version, checkpoint SHA-256, device, threshold), the SHA-256 of the analysed file, warnings and a scope statement. Typed errors carry a message and a machine-readable code.

4.1.7 Storage, Privacy and Security Design

Each analysis has a random 128-bit identifier and a folder holding its status, result and evidence images. The upload itself is written to a temporary file with a server-chosen name, deleted as soon as analysis ends (also on failure), and never used as a path. Results expire after a configurable retention time and can be deleted by the user. Because the identifier is unguessable, a result is reachable only by whoever holds its link; on public deployments the global list endpoint can be switched off so that visitors cannot enumerate each other's analyses, and the dashboard then keeps each browser's own list of identifiers locally. Authentication and rate limiting are *not* implemented (Section 6.1.11).

4.1.8 User Interface and Dashboard Layout

The dashboard has a dark navy and teal theme. The home page (Figure 4.6) contains a single upload card that accepts an image or a video and detects the type from the file, a model-information panel and a list of recent analyses. After an analysis the result page shows the verdict, the model score labelled as an uncalibrated score, and five tabs: Overview, Frame Analysis, Face Tracks, Forensic Evidence and Report (videos), or

Table 4.1: REST endpoints (prefix `/api`).

Method	Path	Purpose
GET	`/health`	Liveness, whether the model is loaded, device
GET	`/model`	Model card, limits, thresholds, Grad-CAM availability
POST	`/analyze/image`	Synchronous image analysis; returns the full result
POST	`/analyze/video`	Starts a background job; returns 202 and an identifier
GET	`/analyses`	Recent analyses (empty on public deployments)
GET	`/analyses/{id}/status`	Job state and progress
GET	`/analyses/{id}`	Full result
GET	`/analyses/{id}/report.pdf`	Forensic report
GET	`/analyses/{id}/artifacts/{name}`	Evidence image (annotated image, crop, heat-map, frame)
DELETE	`/analyses/{id}`	Delete results and cancel a running job

Overview, Forensic Evidence and Report (images). The heat-map viewers are described in Chapter 5.

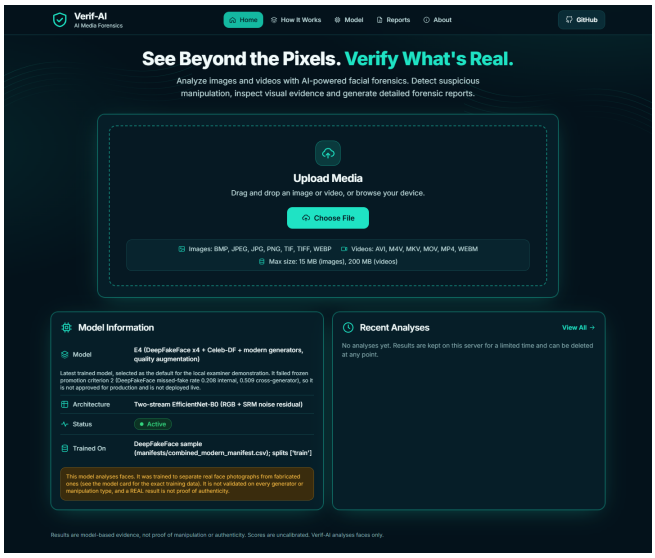


Figure 4.6: Home page of the dashboard (current dark interface; screenshot taken from the running application with an empty history).

4.1.9 Deployment Architecture

The public demonstration separates a static frontend from a hosted API (Figure 4.7). The frontend is a build of the React application served by Vercel; it reads the API address from a build-time environment variable. The API runs in a Docker container on Render and answers browser requests only for the configured frontend origin (CORS). Results are written to the container's disk, which on the free tier is not persistent. A paid instance with at least 2 GB of memory and a disk would be required to run the full model with heat-maps and durable results.

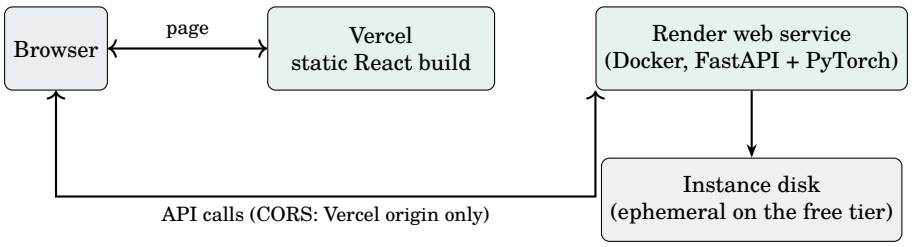


Figure 4.7: Deployment architecture of the public demonstration. No uploaded media is sent to any service other than the API.

Chapter 5

Implementation

5.1 Implementation and Engineering

This chapter describes how the design of Chapter 4 was realised: the environment, the module structure, the algorithms, the training and evaluation tooling, the dashboard, the tests and the deployment. All code is in the project repository; key excerpts are listed in Appendix A.

5.1.1 Development Environment Setup

The backend is a Python package (`backend/verifai`) installed from `requirements.txt`; training and development extras are separate files. Configuration is read from environment variables or an optional `.env` file so that the same code runs locally, in Docker and on a hosting service. The frontend (`frontend/`) is built with Vite and served either by the API process (single-container mode) or by a static host. Typical commands are:

```
cd backend && python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
cd ../frontend && npm ci && npm run build
cd ../backend && set VERIFAI_FRONTEND_DIR=..\frontend\dist
python -m uvicorn verifai.api.app:app --host 127.0.0.1 --port 8000
```

Listing 5.1: Running the project locally.

5.1.2 Algorithm Specifications

The operational logic is governed by three procedures. They make explicit the order of preprocessing and the way scores are aggregated.

Algorithm 1 Image analysis

Input: image file I, settings S, model M
Output: label and score for I, per-face results, evidence images

$rgb \leftarrow \textsc{DecodeAndValidate}(I)$ ▷ format, size, pixel limits
$faces \leftarrow \textsc{DetectFaces}(rgb)$ ▷ MTCNN; keep conf ≥ 0.90, side ≥ 32 px; at most 10
if $faces = \emptyset$ **then**
  return INCONCLUSIVE ▷ no usable face is a result, not an error
end if
$X \leftarrow \textsc{Normalise}(\textsc{JpegRoundTrip}_{q=95}(\textsc{Crop}_{224}(faces)))$
$p \leftarrow M(X)$ ▷ raw sigmoid score per face
$maps \leftarrow \textsc{GradCAM}(M.\text{rgb_stream}, X)$ ▷ skipped if unavailable
$p_{img} \leftarrow \max_i p_i$; $label \leftarrow \textsc{Classify}(p_{img}, \text{margin} = 0.15)$
store annotated image, face crops, heat-maps; **return** $(label, p_{img})$

5.1.3 Module-wise Implementation Breakdown

Face localisation and normalisation. Detection uses the pre-trained MTCNN of [32] through the `facenet-pytorch` package. Detection runs on a downscaled copy for speed;

Table 5.1: Core modules and file structure of the repository.

Module / path	Responsibility
`verifai/config.py`	Settings from environment variables; default model order; limits
`verifai/model.py`	`SRMLayer`, `TwoStreamNet`, safe checkpoint loading, thread-safe prediction
`verifai/preprocess.py`	JPEG round-trip and ImageNet normalisation shared by training and inference
`verifai/faces.py`	MTCNN detection with confidence and size filters; box overlap for tracking
`verifai/registry.py`	Model registry: default / alternative / rollback roles, per-analysis selection, model identification
`verifai/scoring.py`	Labels, INCONCLUSIVE band, decisiveness, aggregation, temporal statistics, suspicious segments
`verifai/image_pipeline.py`, `video_pipeline.py`	Image and video analysis; evidence artifacts
`verifai/explain.py`	Grad-CAM on the RGB stream's last block
`verifai/report.py`	PDF forensic report
`verifai/store.py`, `engine.py`	Result storage, retention, background jobs, cancellation
`verifai/api/app.py`	FastAPI application, upload validation, routes, CORS
`training/`	Data preparation and validation, training, evaluation, degradations, stream ablation, quality bins, acceptance gate, modern-generator data
`frontend/src/`	Pages, result viewers, upload card, history, API client, tests
`kaggle/`	Builders for the private GPU notebooks
`docs/`	Architecture, evaluation, dataset, API, guides and experiment records

Algorithm 2 Video analysis (background job)

Input: video file V, sampling rate ≈ 2 fps, frame cap 64
Output: verdict, tracks, suspicious ranges, evidence frames

$frames \leftarrow$ SampleFrames(V); skip corrupt frames
for each frame f **do**
    $faces_f \leftarrow$ DetectFaces(f); assign each face to a track by box overlap
end for
score all face crops in batches; $s_f \leftarrow \max$ face score in frame f
for every track with ≥ 3 observations: $s_{track} \leftarrow \text{median}$ of its scores
$p_{video} \leftarrow \max_{track} s_{track}$ ▷ pooled fallback when no track is long enough
$ranges \leftarrow$ maximal runs of consecutive frames with $s_f > 0.5$
$E \leftarrow$ six frames with highest s_f; store frame with boxes and Grad-CAM of its top face
return (Classify(p_{video}), $tracks$, $ranges$, E)

boxes are scaled back and crops are cut from the original image. The same detector produced all training crops.

Model loading. Checkpoints are loaded with weights-only deserialisation and strict key matching, so a mismatching file raises an error instead of silently loading partial weights. Both the original prototype's raw state dictionaries and the new files with metadata are accepted. The SHA-256 of the checkpoint is computed at load time and recorded in every result.

Algorithm 3 Selecting what the frame viewer shows

Input: selected frame index i, analysed frames F, stored evidence E (frame image, optional heat-map)

$e \leftarrow E[i]$ if present
if e exists **then**
    show the stored frame (with face boxes); show e.heatmap if it exists, else say "not generated"
else
    say "no frame image or heat-map was stored for this timestamp"; offer the nearest stored frame ▷ nothing is approximated
end if
selecting a timestamp range r: choose the stored evidence frame with the highest score inside r, else the highest-scoring analysed frame

Model registry. The registry knows E4, Model 4 and v2. The server loads one default model at start-up (E4 locally; the hosted image pins v2); another model is loaded on first use and at most one such model stays in memory. `GET /api/models` lists the models with their roles, the analysis endpoints accept a `model` field, and every result, dashboard chip and PDF report records the model name, role, status note and checkpoint hash.

Scoring and aggregation. Labels use the fixed 0.5 threshold and the ± 0.15 reporting band. Aggregation methods (mean, median, weighted mean, suspicious fraction) are implemented; the median is used. Temporal statistics and suspicious ranges are pure functions and are unit tested.

Grad-CAM. The heat-map is generated by the `grad-cam` package on the last block of the RGB stream. It needs gradients, so it runs with autograd enabled under the model lock; it also needs extra memory, which matters on small hosts (Section 5.1.7).

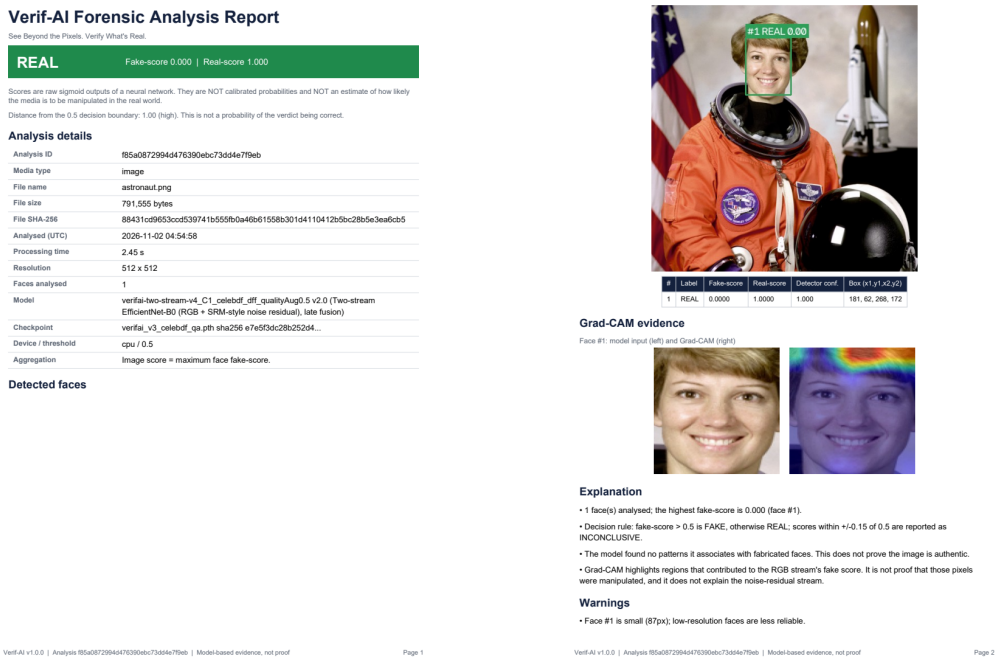
PDF report. The report is built with ReportLab from the stored result and evidence images. It contains the verdict banner, the sentence that scores are not calibrated, analysis details (file hash, model, checkpoint hash, device, threshold), detected faces and heat-maps, video tables where applicable and the limitations (Figure 5.1).

Jobs and storage. A single worker thread pool runs video jobs; a lock serialises model calls. On start-up interrupted jobs are marked as failed, and a periodic clean-up removes results older than the retention time.

5.1.4 Training and Evaluation Implementation

Data preparation. The DeepFakeFace subset was sampled with one fake counterpart per real image, cropped with the production detector and split by *identity and pair* so that no person appears in two splits; exact and perceptual duplicates were removed and the split disjointness was verified by a validation script before training. Celeb-DF videos were split by identity and fakes that cross splits were dropped. For experiment E3 the splits are file-disjoint and near-duplicates of training crops are removed from validation and test with a 64-bit difference hash (Hamming distance ≤ 3).

Training. All models use AdamW [33] with learning rate 10^{-4}, batch size 32, binary cross-entropy with logits (positive-class weight set from the training class counts: 1.02 for the DeepFakeFace runs, 0.82 in E4), a learning-rate schedule that reduces on plateau, gradient clipping at 5, at most 12–15 epochs and early stopping with patience 4. The best epoch is chosen on a *validation* metric only. For the production model it is the validation ROC-AUC; for later experiments with degraded inputs it is the mean of the clean and degraded validation ROC-AUC. Quality augmentation randomly applies blur, down-up resampling, JPEG compression, noise or low-light degradation. Training ran in private Kaggle notebooks on a Tesla T4; only checkpoints and metrics were exported from notebooks that used third-party data.



Verif-AI Forensic Analysis Report

See Beyond the Pixels. Verify What's Real.

REAL Fake-score 0.000 | Real-score 1.000

Scores are raw sigmoid outputs of a neural network. They are NOT calibrated probabilities and NOT an estimate of how likely the media is to be manipulated in the real world.

Distance from the 0.5 decision boundary: 1.00 (high). This is not a probability of the verdict being correct.

Analysis details

Analysis ID	f85a0872994d476390ebc73dd4e7f9eb
Media type	image
File name	astronaut.png
File size	791,555 bytes
File SHA-256	88431cd9653ccd539741b555fb0a46b61558b301d4110412b5bc28b5e3ea6cb5
Analysed (UTC)	2026-11-02 04:54:58
Processing time	2.45 s
Resolution	512 x 512
Faces analysed	1
Model	verifai-two-stream-v4_C1_celebdf_dff_qualityAug0.5 v2.0 (Two-stream EfficientNet-B0 (RGB + SRM-style noise residual), late fusion)
Checkpoint	verifai_v3_celebdf_qa.pth sha256 e7e5f3dc28b252d4...
Device / threshold	cpu / 0.5
Aggregation	Image score = maximum face fake-score.

Detected faces

Verif-AI v1.0.0 | Analysis f85a0872994d476390ebc73dd4e7f9eb | Model-based evidence, not proof Page 1

#	Label	Fake-score	Real-score	Detector conf.	Box (x1,y1,x2,y2)
1	REAL	0.0000	1.0000	1.000	181, 62, 268, 172

Grad-CAM evidence

Face #1: model input (left) and Grad-CAM (right)

Explanation

• 1 face(s) analysed; the highest fake-score is 0.000 (face #1).

• Decision rule: fake-score > 0.5 is FAKE, otherwise REAL; scores within +/-0.15 of 0.5 are reported as INCONCLUSIVE.

• The model found no patterns it associates with fabricated faces. This does not prove the image is authentic.

• Grad-CAM highlights regions that contributed to the RGB stream's fake score. It is not proof that those pixels were manipulated, and it does not explain the noise-residual stream.

Warnings

• Face #1 is small (87px); low-resolution faces are less reliable.

Verif-AI v1.0.0 | Analysis f85a0872994d476390ebc73dd4e7f9eb | Model-based evidence, not proof Page 2

(a) Page 1 (b) Page 2

Figure 5.1: Pages 1 and 2 of a PDF forensic report generated by the application for a public test image. The report records the verdict, the uncalibrated-score statement, the file and checkpoint hashes and the detected face. (In this version the “Detected faces” heading is separated from its image by a page break, a minor layout flaw.)

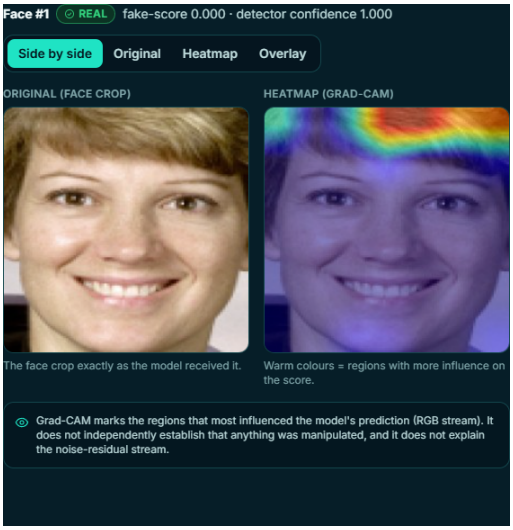
Evaluation tooling. The evaluation script reports accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrices and error rates with grouped-bootstrap intervals, per generator and per split, at the fixed threshold, optionally under class-blind degradations; further tools measure image sharpness bins, stream-level behaviour, an acceptance gate and video-level metrics.

5.1.5 Frontend Implementation

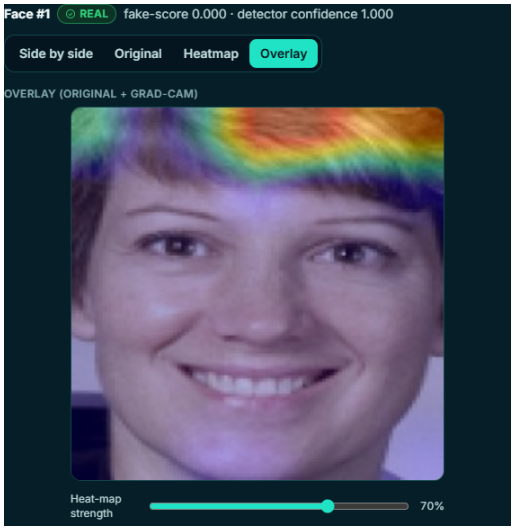
The dashboard is a React/TypeScript single-page application with hash routes, so a refresh or a deep link works on any static host. The upload card validates the file type and size, detects images versus videos from the extension, shows a preview and calls the image or the video endpoint. The result page is assembled only from fields of the API response; nothing is mocked or hard-coded.

Heat-map viewer for images. For each detected face the viewer shows the original face crop and the Grad-CAM image side by side, or one of them alone, or an *overlay* that blends the Grad-CAM image over the crop with an adjustable strength (Figure 5.2). Both images are artifacts produced by the server; the interface does not compute or approximate a heat-map. If several faces are detected a selector chooses which face is shown, and each face’s verdict, score and detector confidence are displayed with it.

Frame viewer for videos. The timeline plots one dot per analysed frame; a dot is selectable. The selected frame’s stored image and heat-map come from the same evidence entry, so they cannot drift apart. Frames without stored evidence show an explicit notice and a link to the nearest stored frame (Figure 5.3). Selecting a suspicious time range jumps to the highest-scoring frame in it. The overlay mode is disabled for video because the server stores the heat-map of the face crop but not its exact position in the frame, and a registered overlay would be an approximation.



(a) Side by side



(b) Overlay (70% strength)

Figure 5.2: Heat-map viewer for an image result (public test image). The viewer also offers Original-only and Heatmap-only views. The text under the viewer states that Grad-CAM marks regions that influenced the model and does not establish manipulation.

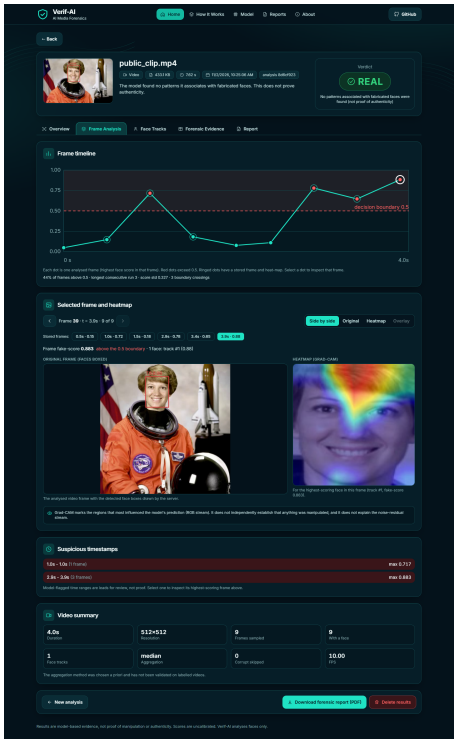
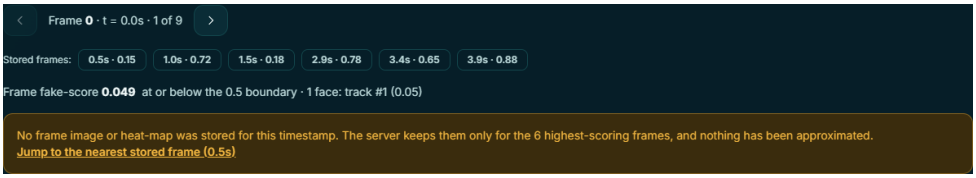
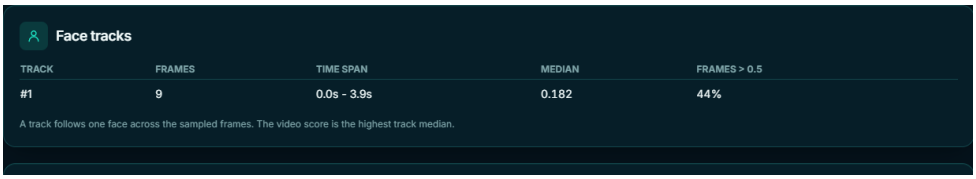


Figure 5.3: Frame Analysis tab for a video (a synthetic clip made from the public test image): the clickable timeline with the 0.5 decision boundary, the selected frame with the face boxes and its Grad-CAM, and the stored-frame shortcuts.

History and deletion. Because a public API should not list other visitors' analyses, the dashboard stores the identifiers of the analyses created in that browser and asks the server for each one; deleted or expired identifiers are dropped. Deleting a result removes the server-side folder and the local entry.



(a) Frame without stored evidence



(b) Face Tracks tab (the card shown on the Face Tracks tab)

Figure 5.4: Honest unavailability and face tracks. (a) The server keeps frames and heat-maps only for the six highest-scoring frames; other frames say so. (b) Tracks with frame counts, time span, median score and the share of frames above 0.5.

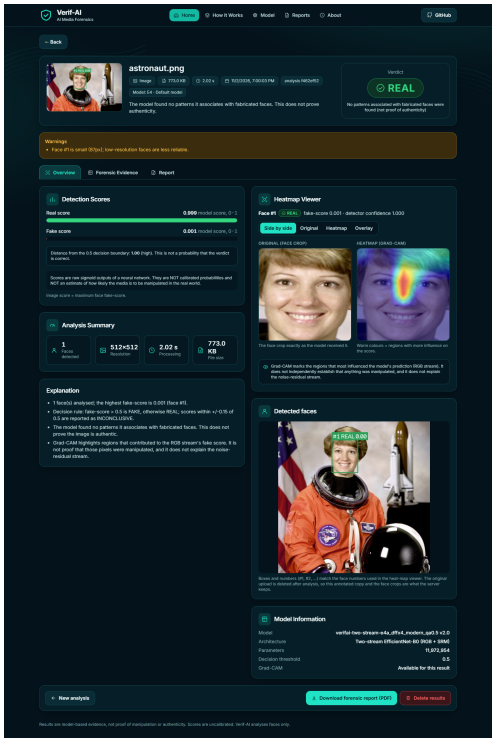


Figure 5.5: Overview tab of an image result: verdict, model score labelled as uncalibrated, analysis summary, the heat-map viewer and the detected faces with bounding boxes. The "Trained On" text in the model panel (Figure 4.6) is the checkpoint's own metadata, which understates the data used for this model (Section 6.1.11).

5.1.6 Testing and Verification

Verification was automated wherever possible. The backend suite covers model loading, data validation, scoring and tracking, the image and video pipelines, the API, the PDF report, training smoke tests and the research tooling; at the time of writing 152 tests passed and 7 were skipped because they need a test video, a GPU or private media (the skips are environment-gated and were not hidden). The frontend has 63 unit and component tests (including the viewer modes, frame synchronisation and the unavailable states) and 9 Playwright end-to-end tests that drive a real Chrome browser: unified

upload, image and video analysis, heat-map modes, timeline selection, PDF download, deletion, rejected files, navigation and refresh, console errors and layouts from 1440 to 320 pixels wide. The end-to-end suite was run against the local server, against a separate frontend origin and against the public deployment.

5.1.7 Deployment Implementation

Frontend. The frontend is deployed on Vercel as a static build; `vercel.json` fixes the build command and output directory and adds security headers, and the API address is the build-time variable `VITE_API_BASE_URL`. Direct navigation and a refresh were verified on the public URL.

Backend. The API runs in a Docker container on Render's free tier with CORS limited to the Vercel origin. The memory needed by the backend was measured on the development laptop (Table 5.2).

Table 5.2: Backend memory (process working set, MB) measured on the development laptop for different configurations.

Configuration	Idle	Image peak	Video peak
Default (Grad-CAM on, batch 16)	491	776	over 818
Grad-CAM on, batch 2, one thread	488	672	1473
Grad-CAM off, batch 2, one thread	488	510	576

A 512 MB instance cannot run the default configuration: the first analysis on the free tier returned HTTP 502 (the process was killed for lack of memory). The public instance therefore runs in a restricted "lite" mode with Grad-CAM disabled, small batches, a single thread and small upload limits; the interface reports that heat-maps are unavailable instead of failing. It serves the model checkpoint contained in the repository (v2) because the newer checkpoint is not stored in Git and the free tier has no persistent disk. Results disappear when the instance restarts and the first request after an idle period takes about a minute. **This deployment is a restricted demonstration, not a production system;** full functionality needs at least 2 GB of memory and a disk, authentication and rate limiting.

5.1.8 Engineering Challenges and Optimisations

- **Identical preprocessing.** Training crops had been stored as JPEG, so inference reproduces the JPEG round-trip; without it the scores differ from the evaluated ones.
- **Leakage control.** Identity-disjoint splits and duplicate removal were implemented before training; later experiments add near-duplicate removal across training, validation and test (179 validation and 373 test crops were removed in E3).
- **Memory.** The memory profile above led to configurable batch sizes, optional Grad-CAM and the lite mode.
- **Privacy.** Deleting uploads in all code paths, expiring results and removing the public history list required changes in both backend and frontend.
- **Third-party data.** Notebooks that use third-party images export only checkpoints and metrics; a failed run's output can contain working files, so logs were read through the API without downloading outputs.

Chapter 6

Results and Discussion

6.1 Results, Evaluation and Discussion

This chapter presents the experiments in the order in which they were carried out, because each one explains the next: the production model and its measured performance, a failure on genuine video, the diagnosis of that failure as an image-quality shortcut, an acceptance gate, a retraining that fixed the video failure at a cost, and an experiment with modern generators. Every number comes from a saved result file of the repository; the limits of each number are stated next to it.

6.1.1 Experimental Setup and Dataset Partitioning

DeepFakeFace (DFF). A sample of the dataset [25] was used. The real images are IMDB-WIKI photographs [26]; for each real image one fake counterpart is generated by inpainting, face swapping ("insight") or a text-to-image model ("text2img"). Splits are identity- and pair-disjoint after duplicate removal (Table 6.1). The text-to-image fakes occur only in the cross-generator test split, so that split measures behaviour on an *unseen generator*.

Table 6.1: DeepFakeFace splits used for the production model (face crops).

Split	Role	n (real / fake)	Fake generators
train	fitting	2,315 (1,170 / 1,145)	inpainting, insight
val	model selection	471 (239 / 232)	inpainting, insight
test_internal	unseen identities, seen generators	575 (292 / 283)	inpainting, insight
test_crossgen	unseen identities and unseen generator	568 (281 / 287)	text2img

Celeb-DF v2. Videos [19] were split by identity into training, validation and held-out test identities (nine test identities, 147 videos: 91 real and 56 fake); fakes whose source and target identities fall in different splits were dropped. Frames were sampled from each video, cropped with the production detector and stored as JPEG. Test results are reported at frame level and at video level.

Experiment E3 data. Modern-generator data were added from FFHQ real faces [2], SFHQ synthetic faces [27] and the fake side of the DeepDetect-2025 collection (Table 6.2). The datasets were reviewed for provenance and shortcuts before use (Appendix C); two candidate datasets were excluded.

Table 6.2: Composition of the experiment E3 manifest after face detection and near-duplicate removal (crops).

Source (label)	Train	Val.	modern_test	Notes
FFHQ (real)	7,996	966	1,447	file-disjoint, seed 42
SFHQ (fake)	3,000	473	677	StyleGAN-generated
DeepDetect fake	8,997	880	2,202	modern generators; no per-image generator label
DeepFakeFace + Celeb-DF	as for Model 4	as before	separate tests	original splits unchanged

6.1.2 Performance Evaluation Methodology

Metrics. With "fake" as the positive class the report uses accuracy, precision, recall, F1, the area under the ROC curve (ROC-AUC), the false-positive rate (FPR: genuine faces called fake) and the false-negative rate (FNR: fakes called genuine), together with confusion matrices. Intervals are Wilson 95% intervals [35] for rates and grouped-bootstrap intervals [36] for the other metrics.

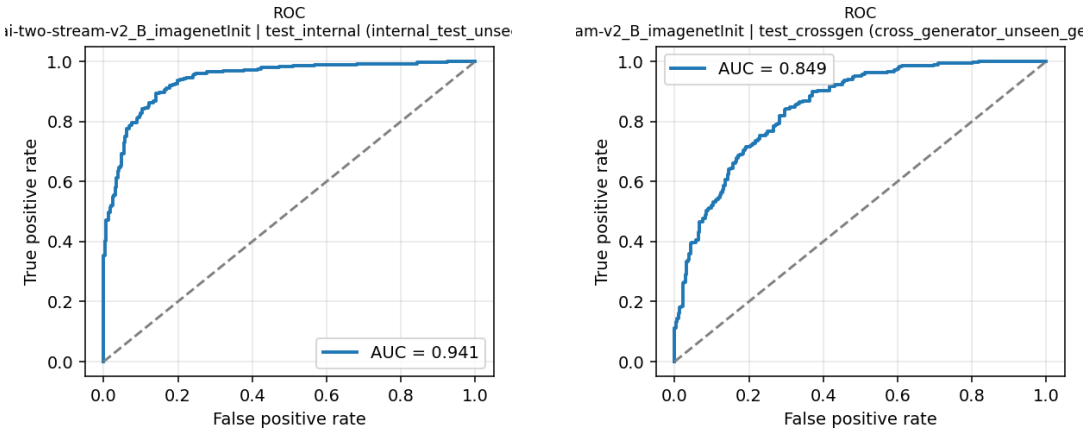
Rules fixed in advance. (i) The decision threshold is 0.5 and is never tuned. (ii) Model selection uses *validation* data only; test data are never used for selection or for thresholds. (iii) Results on held-out identities with seen generators (*internal*) are always reported separately from results on an unseen generator (*cross-generator*). (iv) Celeb-DF results are *within-dataset* evidence: the held-out identities share the face-swap method and the footage type of the training videos, so they say nothing about other manipulation methods. (v) Scores are raw sigmoid outputs, not probabilities.

6.1.3 Results for the Production Model v2 (DeepFakeFace)

The production model v2 was chosen among three training configurations by validation ROC-AUC alone (ImageNet initialisation, 9th epoch, validation ROC-AUC 0.948); its test numbers were never consulted for selection. Table 6.3 shows its results.

Table 6.3: Production model v2 on DeepFakeFace (threshold 0.5; fake is positive; intervals are 95% grouped bootstrap).

Split	Acc	Prec	Recall	F1	ROC-AUC	FPR / FNR
Validation (selection)	0.877	0.843	0.922	0.881	0.948 (0.928–0.965)	0.167 / 0.078
Internal test	0.875 (0.848–0.902)	0.860	0.890	0.875	0.941 (0.921–0.960)	0.140 / 0.110
Cross-generator test	0.746 (0.716–0.778)	0.818	0.641	0.719	0.849 (0.818–0.879)	0.146 / 0.359

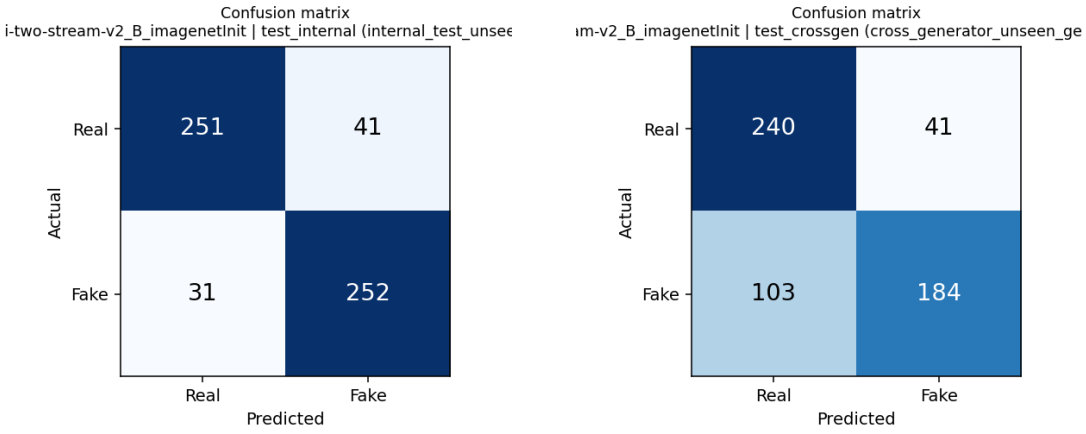


(a) Internal test

(b) Cross-generator test

Figure 6.1: ROC curves of v2 on the two held-out DeepFakeFace test splits (generated by the evaluation script).

On the internal split v2 detects inpainting fakes 91.5% and face swaps 86.6% of the time and accepts 85.9% of real photographs. On the unseen text-to-image generator the detection rate falls to 64.1%. This gap between internal and cross-generator performance is the first evidence that the measured accuracy depends on the generators seen during training, and *the internal numbers must not be quoted as general accuracy*. For comparison, the original prototype's checkpoint, evaluated on the same identity-disjoint data, performed at chance level; the very high accuracy reported for the prototype could therefore not be reproduced under this protocol.



(a) Internal test

(b) Cross-generator test

Figure 6.2: Confusion matrices of v2 at threshold 0.5. On the unseen generator the model misses a much larger share of fakes (103 of 287).

6.1.4 A Failure on Genuine Video: the Four-Model Comparison

When genuine video of a person was analysed with v2, the application sometimes reported FAKE with very high scores. To test this systematically, four models were evaluated with identical code on the held-out Celeb-DF identities: Model 1 (production v2), Model 2 (DeepFakeFace only with quality augmentation), Model 3 (DeepFakeFace plus Celeb-DF) and Model 4 (as Model 3 with quality augmentation). Table 6.4 gives the video-level results.

Table 6.4: Video-level results on 147 held-out Celeb-DF videos (91 real, 56 fake), clean inputs, 95% Wilson intervals.

Model	**Accuracy**	**FPR (real called fake)**	**FNR (fake called real)**
1 Production v2	0.483 (0.404–0.563)	55/91 = 0.604 (0.502–0.699)	21/56 = 0.375 (0.260–0.506)
2 DFF only + quality aug.	0.578 (0.497–0.655)	16/91 = 0.176 (0.111–0.267)	46/56 = 0.821 (0.702–0.900)
3 DFF + Celeb-DF	0.973 (0.932–0.989)	4/91 = 0.044 (0.017–0.108)	0/56 = 0.000 (0.000–0.064)
4 DFF + Celeb-DF + quality aug.	0.980 (0.942–0.993)	2/91 = 0.022 (0.006–0.077)	1/56 = 0.018 (0.003–0.094)

The production model is at chance level on this video set (55 of 91 genuine videos called fake). Models trained on Celeb-DF frames reach 0.97–0.98 video accuracy, but this is *within-dataset* evidence: the test identities share the face-swap method and the footage type of the training videos. Adding Celeb-DF also had a cost on DeepFakeFace: Model 4's false-negative rate on the unseen generator rose from 0.359 (v2) to 0.634, and on the internal test from 0.110 to 0.201 (Table 6.6).

6.1.5 Diagnosis: an Image-Quality Shortcut

Experiment E1 measured whether image sharpness alone predicts the label (no model involved). In DeepFakeFace the fakes are softer than the real photographs: the ROC-AUC of *softness alone* as a classifier of FAKE is 0.712 (95% CI 0.696–0.729) on the training split, 0.680 on the internal test split and — on the unseen generator — **0.432**, i.e. the cue reverses. The share of fakes among crops rises from 26% in the sharpest bin to 85% in the softest (Figure 6.3). Celeb-DF frames are almost all very soft (about 94% fall into the softest sharpness bin, against about 8% of DeepFakeFace training crops), so a model trained on DeepFakeFace alone has seen almost no soft real faces and learns "soft face means fake", which explains the false alarms on soft genuine video. Stream-level analysis showed that the noise stream carries this dependence: the Spearman correlation between sharpness and the noise-stream score was -0.36 (real faces) and -0.52 (fake

faces) for the production model on the DeepFakeFace internal test split, while the RGB stream's correlation was close to zero. On the Celeb-DF validation faces the production model's noise stream alone called 98.7% of the real faces fake (stage-0 diagnostics).

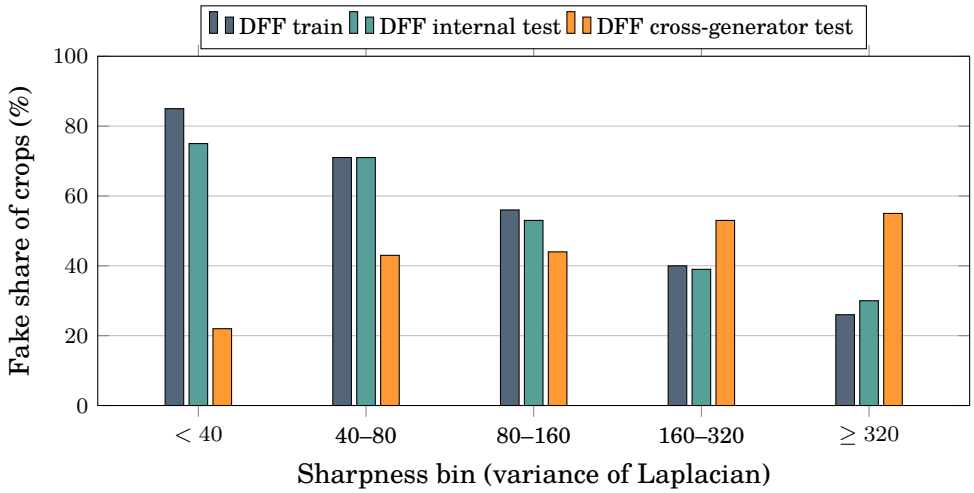


Figure 6.3: Fake share per fixed sharpness bin in DeepFakeFace (values from the E1 diagnostic). In the training and internal splits softness predicts "fake"; on the unseen generator the relation is reversed, which shows that sharpness is a dataset shortcut and not a manipulation cue.

6.1.6 The Stage-0 Acceptance Gate

Because overall accuracy hides this kind of failure, a per-quality-bin acceptance gate was designed and its thresholds were frozen in a protocol document before any run. It checks that each stream separates the classes on its own (G1), that scores are not correlated with sharpness within a class (G2), that error rates do not vary across quality bins (G3) and that the model separates the classes within bins (G4), on validation data only, clean and with class-blind degradations. Synthetic controls validate the gate itself: a softness-only classifier must fail it and a quality-independent scorer must not. All four models failed the gate (Table 6.5); models 3 and 4 pass the ungated screen (ROC-AUC ≥ 0.90 on both clean validation sets) yet fail the gate, which shows that the gate carries information that the aggregate metric does not. The thresholds were chosen while reading E1 results that included test splits, so stage-0 cannot prove independence of its thresholds from the test data; this is stated as a limitation.

Table 6.5: Stage-0 gate results (validation data, clean and degraded sets; DFF = DeepFakeFace, CDF = Celeb-DF).

Model	**Passes ungated screen**	**Gate failures (set:check)**
1 Production v2	no	13 failures across DFF and CDF, clean and degraded
2 DFF only + quality aug.	no	9 failures
3 DFF + Celeb-DF	yes	DFF-clean:G3; DFF-degraded:G1–G4; CDF-degraded:G2–G4
4 DFF + Celeb-DF + quality aug.	yes	DFF-clean:G2; DFF-degraded:G1; CDF-degraded:G2

6.1.7 Experiment E3: Modern-Generator Training

The protocol of E3 (data, leakage control, candidates, evaluation, promotion rule) was written *before* training. Two candidates were trained on the enlarged data with the Model-4 recipe: E3A with quality augmentation and E3B without. The choice between

them used validation only (E3A, validation selection score 0.980 against 0.939). Both were evaluated with Model 4 and v2 under the same protocol (Tables 6.6–6.9).

Reading the tables. The "Validation" row is the *combined* E3 validation split, which includes modern-generator and Celeb-DF crops that Model 4 and v2 were never trained on; their low accuracy there reflects that mismatch and is not a regression. The E3 evaluation re-extracted the Celeb-DF crops, so the Celeb-DF numbers differ slightly from the earlier four-model comparison (Model 4: video accuracy 0.980 in both; v2: 0.476 here against 0.483 there).

Table 6.6: Frame-level results, clean inputs, fixed threshold 0.5 (FPR: real called FAKE; FNR: fake called REAL; 95% Wilson intervals). Counts are real / fake crops.

Split	Model	n (R/F)	Acc	AUC	FPR	FNR
Validation	E3A	2045/2410	0.952	0.990	0.039 [0.03,0.05]	0.056 [0.05,0.07]
	E3B	2045/2410	0.954	0.992	0.056 [0.05,0.07]	0.037 [0.03,0.05]
	Model 4	2045/2410	0.654	0.713	0.066 [0.06,0.08]	0.583 [0.56,0.60]
	v2	2045/2410	0.489	0.484	0.319 [0.30,0.34]	0.675 [0.66,0.69]
DFF internal test	E3A	292/283	0.833	0.916	0.106 [0.08,0.15]	0.230 [0.18,0.28]
	E3B	292/283	0.861	0.939	0.130 [0.10,0.17]	0.148 [0.11,0.19]
	Model 4	292/283	0.835	0.921	0.130 [0.10,0.17]	0.201 [0.16,0.25]
	v2	292/283	0.875	0.941	0.140 [0.11,0.18]	0.110 [0.08,0.15]
DFF cross-generator test	E3A	281/287	0.643	0.793	0.085 [0.06,0.12]	0.624 [0.57,0.68]
	E3B	281/287	0.729	0.850	0.089 [0.06,0.13]	0.449 [0.39,0.51]
	Model 4	281/287	0.627	0.795	0.107 [0.08,0.15]	0.634 [0.58,0.69]
	v2	281/287	0.746	0.849	0.146 [0.11,0.19]	0.359 [0.31,0.42]
Celeb-DF test (frames)	E3A	1453/895	0.954	0.989	0.046 [0.04,0.06]	0.045 [0.03,0.06]
	E3B	1453/895	0.957	0.991	0.050 [0.04,0.06]	0.032 [0.02,0.05]
	Model 4	1453/895	0.953	0.988	0.038 [0.03,0.05]	0.061 [0.05,0.08]
	v2	1453/895	0.471	0.479	0.593 [0.57,0.62]	0.425 [0.39,0.46]
modern_test	E3A	1447/2879	0.983	0.998	0.014 [0.01,0.02]	0.019 [0.01,0.02]
	E3B	1447/2879	0.981	0.998	0.037 [0.03,0.05]	0.011 [0.01,0.02]
	Model 4	1447/2879	0.328	0.414	0.082 [0.07,0.10]	0.969 [0.96,0.97]
	v2	1447/2879	0.326	0.398	0.090 [0.08,0.11]	0.968 [0.96,0.97]

Table 6.7: Video-level results on the Celeb-DF held-out identities (91 real and 56 fake videos), clean inputs.

Model	Accuracy	ROC-AUC	FPR [95% CI]	FNR [95% CI]
E3A	0.980	1.000	0.022 [0.01,0.08]	0.018 [0.00,0.09]
E3B	0.980	0.999	0.022 [0.01,0.08]	0.018 [0.00,0.09]
Model 4	0.980	0.998	0.011 [0.00,0.06]	0.036 [0.01,0.12]
v2	0.476	0.479	0.604 [0.50,0.70]	0.393 [0.28,0.52]

Table 6.8: Share of crops called FAKE per source, clean inputs (for real sources this is the false-positive rate; for fake sources it is the detection rate).

Split	Source	n	E3A	E3B	Model 4	v2
modern_test	deepdetect_fake	2202	0.996 [0.99,1.00]	0.997 [0.99,1.00]	0.029 [0.02,0.04]	0.028 [0.02,0.04]
	real_ffhq	1447	0.014 [0.01,0.02]	0.037 [0.03,0.05]	0.082 [0.07,0.10]	0.090 [0.08,0.11]
	sfhq	677	0.931 [0.91,0.95]	0.963 [0.95,0.97]	0.038 [0.03,0.06]	0.044 [0.03,0.06]
test_internal	inpainting	141	0.780 [0.70,0.84]	0.837 [0.77,0.89]	0.787 [0.71,0.85]	0.915 [0.86,0.95]
	insight	142	0.761 [0.68,0.82]	0.866 [0.80,0.91]	0.810 [0.74,0.87]	0.866 [0.80,0.91]
	real	292	0.106 [0.08,0.15]	0.130 [0.10,0.17]	0.130 [0.10,0.17]	0.140 [0.11,0.18]
test_crossgen	real	281	0.085 [0.06,0.12]	0.089 [0.06,0.13]	0.107 [0.08,0.15]	0.146 [0.11,0.19]
	text2img	287	0.376 [0.32,0.43]	0.551 [0.49,0.61]	0.366 [0.31,0.42]	0.641 [0.58,0.69]
cdf_test	celebdf_synthesis	895	0.955 [0.94,0.97]	0.968 [0.95,0.98]	0.939 [0.92,0.95]	0.575 [0.54,0.61]
	real	1453	0.046 [0.04,0.06]	0.050 [0.04,0.06]	0.038 [0.03,0.05]	0.593 [0.57,0.62]

Table 6.9: Robustness to a class-blind *mixed* degradation (blur, down-up resampling, JPEG, noise, low light) applied identically to every crop.

Model	modern_test Acc	FPR	FNR	Celeb-DF video Acc	video FPR
E3A	0.945	0.080	0.043	0.973	0.033
E3B	0.767	0.241	0.229	0.973	0.044
Model 4	0.343	0.142	0.915	0.973	0.011
v2	0.414	0.397	0.680	0.388	0.725

Table 6.10: The frozen promotion rule applied to E3A (rule written before training).

#	Criterion	Value	Result
1	Celeb-DF video FPR $\leq$ Model 4's + 0.03 and accuracy ≥ 0.95	FPR 0.022 vs 0.011; acc 0.980	**PASS**
2	DFF test_internal: FNR $\leq$ v2's + 0.05 and FPR ≤ 0.20	FNR 0.230 (limit 0.160); FPR 0.106	**FAIL**
2	DFF test_crossgen: FNR $\leq$ v2's + 0.05 and FPR ≤ 0.20	FNR 0.624 (limit 0.409); FPR 0.085	**FAIL**
3	modern_test balanced accuracy $\geq$ Model 4's + 0.10, interval above 0	0.984 vs 0.475 (diff +0.509, lower bound +0.500)	**PASS**

Interpretation.

- On `modern_test` E3A and E3B detect almost all held-out modern fakes (about 98% accuracy, FPR 0.014 and 0.037) whereas Model 4 and v2 detect only a few percent of them. The improvement is large and the intervals do not overlap.
- **This is not generator-disjoint evidence.** The fakes in `modern_test` come from the same generator families as the training fakes (DeepDetect carries no per-image generator labels), and its real faces are all FFHQ, so a "FFHQ-versus-generated" style shortcut cannot be excluded. The only unseen-generator test is the DeepFakeFace cross-generator split, where E3A is not better than Model 4 (FNR 0.624 versus 0.634) and clearly worse than v2 (0.359).
- Celeb-DF video accuracy is unchanged at 0.980; the noise-robustness of E3A under the mixed degradation is good on `modern_test`, whereas E3B degrades strongly.
- By the frozen promotion rule (Table 6.10) E3A fails criterion 2 (DeepFakeFace detection must not be more than 0.05 worse than v2). The same criterion fails for Model 4. **E3A was therefore not promoted.**

6.1.8 Experiment E4: Source-Balanced Variant

Experiment E4 repeats every DeepFakeFace *training* row four times (validation and test unchanged) in the E3A recipe (ImageNet initialisation, quality augmentation 0.5, AdamW 10^{-4}, batch 32, at most 12 epochs, seed 42), to test whether source imbalance (DeepFakeFace is about 5% of the training fakes) causes the loss of DeepFakeFace detection. The protocol and the promotion rule were registered before training. The run took 134.5 minutes on the Kaggle T4: all 12 epochs ran and the best epoch (by validation selection score) was epoch 12 (0.9797; validation accuracy 0.954, ROC-AUC 0.990), so training had not plateaued. Table 6.11 gives the held-out test results (same splits and evaluation code as E3) and Table 6.12 applies the promotion rule.

Interpretation. (i) Repeating the DeepFakeFace rows recovered *part* of the unseen-generator detection (FNR 0.509 against 0.634 for Model 4 and 0.624 for E3A) and slightly improved the internal test (FNR 0.208 against 0.230 for E3A), but it did not reach v2 (0.110 and 0.359). (ii) Celeb-DF video accuracy (0.980) and `modern_test` accuracy (0.983) are the same as for E3A. (iii) By the frozen rule E4 passes criteria 1 and 3 but **fails criterion 2** (internal FNR 0.208 against a limit of 0.160; cross-generator FNR 0.509 against 0.409), so **E4 did not pass the project's production-promotion**

Table 6.11: Experiment E4 against Model 4 and v2: held-out results, clean inputs, fixed threshold 0.5 (accuracy/ROC-AUC/FPR/FNR).

Test set	E4	Model 4	v2
DFF internal test (frames)	0.831/0.908/0.130/0.208	0.835/0.921/0.130/0.201	0.875/0.941/0.140/0.110
DFF cross-generator test (frames)	0.688/0.799/0.110/0.509	0.627/0.795/0.107/0.634	0.746/0.849/0.146/0.359
Celeb-DF test (frames)	0.945/0.985/0.061/0.045	0.953/0.988/0.038/0.061	0.471/0.479/0.593/0.425
Celeb-DF test (**video**)	0.980/0.999/0.022/0.018	0.980/0.998/0.011/0.036	0.476/0.479/0.604/0.393
modern_test (frames)	0.983/0.998/0.023/0.014	0.328/0.414/0.082/0.969	0.326/0.398/0.090/0.968

Table 6.12: The frozen promotion rule applied to E4 (rule written before training).

#	Criterion	Value	Result
1	Celeb-DF video FPR $\leq$ Model 4's + 0.03 and accuracy ≥ 0.95	FPR 0.022 vs 0.011; acc 0.980	**PASS**
2	DFF test_internal: FNR $\leq$ v2's + 0.05 and FPR ≤ 0.20	FNR 0.208 (limit 0.160); FPR 0.130	**FAIL**
2	DFF test_crossgen: FNR $\leq$ v2's + 0.05 and FPR ≤ 0.20	FNR 0.509 (limit 0.409); FPR 0.110	**FAIL**
3	modern_test balanced accuracy $\geq$ Model 4's + 0.10, interval above 0	0.982 vs 0.475 (diff +0.507, lower bound +0.498)	**PASS**

rule. The earlier caveats still apply: `modern_test` is not generator-disjoint, Celeb-DF is within-dataset evidence, and E4 is a single run (one seed).

Final model configuration. E4 has the same architecture as every other model (two EfficientNet-B0 streams, 11,972,954 parameters; Chapter 4) and the same preprocessing, threshold and scoring; only its training data and recipe differ. It is the latest trained model and combines the strongest held-out results on Celeb-DF video (0.980) and `modern_test` (0.983) with a smaller DeepFakeFace loss than Model 4 on the unseen generator, but it does not beat every model on every split: v2 remains best on DeepFakeFace (0.875 and 0.746). By decision of the project owner, **E4 is the default model of the local examiner demonstration**; Model 4 is an alternative, selectable in the dashboard, and v2 is the rollback model. Because E4 did not pass the promotion rule it has *not* replaced v2 on the public deployment: the free-tier host (512 MB, no disk) cannot hold the checkpoint or two models, so the live API still serves v2. Results and PDF reports record the model name, role and checkpoint hash.

6.1.9 Qualitative Analysis of Grad-CAM Output

The heat-maps in Figures 5.2 and 5.3 show typical behaviour for a genuine public-domain portrait: the activation concentrates on the forehead and hairline area and is weak over the central face. This kind of localisation is an *explanation of what the RGB stream used*; it does not show where a manipulation happened and it says nothing about the noise stream. Because both the quality shortcut and the generator-specific cues are present in the training data, a plausible-looking heat-map cannot by itself be taken as evidence that the model uses manipulation artefacts.

6.1.10 Verification of the Deployed System

The end-to-end suite of nine browser tests passed against the local server, against a separately hosted frontend origin (which also verified cross-origin access, PDF download and deletion), and against the public deployment (Vercel frontend and Render API). On the public deployment, image analysis and video analysis with the public test media completed, the verdict, scores, face tracks, timeline selection, PDF download and deletion worked, direct navigation and a page refresh kept the same page, and no console errors occurred. With Grad-CAM switched off on the free tier the viewers showed the explicit "not available" state. The public API returned the v2 checkpoint (hash prefix

3b947755) and an empty history list, as configured. The public deployment is a restricted demonstration; Section 5.1.7 lists its limits.

Informal checks with private media. After the models were frozen, a small set of private demonstration files (genuine and AI-generated photographs and videos) was scored locally as a sanity check. They were never used for training, threshold choice or model selection, were never uploaded, and are *not* reported here for privacy; the set is too small and uncontrolled to count as an evaluation, and one file was a near-duplicate of a training crop.

6.1.11 Limitations and Threats to Validity

- **Face-level, image-trained model.** The model analyses cropped faces; video is handled by aggregating frame scores with an unvalidated rule. No labelled modern AI-video data were available, so **nothing is claimed about AI-generated video.**
- **Generator and source overlap.** Internal splits share generators with the training data; `modern_test` shares generator families and has a single real source. The unseen-generator evidence rests on one dataset split (text-to-image faces from DeepFakeFace).
- **Shortcuts.** Sharpness is a measured shortcut; resolution, file format and the source of the real class can also act as shortcuts, and only some were controlled. Quality augmentation reduced but did not remove the dependence.
- **Celeb-DF.** Results are within-dataset and use one manipulation method.
- **Calibration.** Scores are raw sigmoid outputs and must not be read as probabilities; the INCONCLUSIVE band is a reporting rule, not a calibration.
- **Explanations.** Grad-CAM indicates influential regions of the RGB stream; it does not prove manipulation.
- **Real photographs.** Real images come from IMDB-WIKI, FFHQ and YouTube footage; modern phone photographs, webcams and heavy compression are under-represented, and demographic bias was not characterised.
- **Model metadata.** The checkpoint metadata of Model 4 and E4 lists DeepFakeFace as training data although Celeb-DF was also used; checkpoints are kept unchanged.
- **Deployment.** The public instance runs a different (older) checkpoint without heat-maps, has no persistent storage, no authentication and no rate limiting, and a 512 MB limit makes long videos unreliable. It is not production-ready.
- **Threshold of the gate.** The stage-0 thresholds were set with knowledge of earlier test-split diagnostics (disclosed in its protocol).

6.1.12 Summary of Results

1. The two-stream detector works well on the data it was trained on (v2: accuracy 0.875 internal), degrades on an unseen generator (0.746) and failed on genuine video.
2. The failure was traced to an image-quality shortcut; adding Celeb-DF video and quality augmentation (Model 4) reached 0.980 video accuracy on held-out Celeb-DF identities, at the price of weaker DeepFakeFace detection.
3. Adding modern-generator data (E3) lets the model detect held-out fakes of those generators, but without a generator-disjoint test this is not evidence of general detection, and it did not meet the pre-registered promotion rule; the source-balanced variant E4 recovered only part of the DeepFakeFace detection and did not meet it either.
4. No result supports a claim of 95% (or any fixed) accuracy for modern AI-generated images or videos in general.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion and Future Perspectives

7.1.1 Project Summary and Key Contributions

Verif-AI is a working system for face-based deepfake detection and forensic analysis. It locates faces with MTCNN, scores them with a two-stream network (an RGB EfficientNet-B0 and an EfficientNet-B0 that reads a fixed three-kernel SRM noise residual), explains the score with Grad-CAM heat-maps, summarises videos with face tracks, timelines and stored evidence frames, and produces a PDF forensic report. It is delivered as a tested FastAPI backend and a responsive React dashboard in which uploads are deleted immediately and results can be deleted by the user. Beyond the software, the project contributes an evaluation record that treats failures as results: the chance-level behaviour of the production model on genuine video was reproduced, traced to an image-quality shortcut and partly repaired; a per-quality-bin acceptance gate was designed and validated with synthetic controls; and an experiment with modern-generator data was run under a protocol and a promotion rule that were fixed before training.

7.1.2 Final Performance Assessment

The honest summary of the measured performance is the following. The production model v2 reaches accuracy 0.875 and ROC-AUC 0.941 on held-out identities of DeepFakeFace with generators seen in training, but only 0.746 and 0.849 on an unseen text-to-image generator, and it is at chance level (accuracy 0.483) on held-out Celeb-DF video. The model with Celeb-DF data and quality augmentation (Model 4), the previous default and now an alternative, reaches 0.980 video accuracy on held-out Celeb-DF identities — within-dataset evidence for one face-swap method — but detects fewer DeepFakeFace fakes than v2. The E3 models detect held-out fakes of the modern generator families they were trained on almost perfectly, but this is not generator-disjoint evidence and the models (E3A, and the source-balanced E4) did not meet the pre-registered promotion rule. No result supports a claim of 95% accuracy for modern AI-generated images or videos in general, and none supports a claim about AI-generated video.

7.1.3 Critical Reflections and Limitations

The central lesson is methodological. The prototype's very high reported accuracy could not be reproduced on identity-disjoint data, an unseen generator exposed a large drop, and a cue as simple as sharpness explained much of the apparent skill on the original dataset. Scores are not probabilities, Grad-CAM maps are explanations of a model and not proof, and the system depends on face detection and on the data it was trained on. The public demonstration runs on free hosting with a smaller, older model, without heat-maps and without persistent storage, and has no authentication or rate limiting; it is a demonstration, not a production service. Section 6.1.11 lists the limitations in detail.

7.1.4 Future Perspectives

- **Independent evaluation data.** Assemble labelled test sets with *generator-disjoint* and *source-disjoint* splits, genuine phone and webcam photographs and video, and, if licences allow, labelled AI-generated video; only then can claims about modern generators be tested.
- **Shortcut-robust training.** Balance training data across quality bins and sources, supervise each stream separately, and report the per-bin gate for every candidate; the source-balanced E4 experiment (Section 6.1.8) is a first step in this direction.
- **Calibration and uncertainty.** Calibrate scores on held-out data of the intended use and report calibrated uncertainty.
- **Stronger explanation.** Store heat-maps for every analysed frame together with their registered position so that a true overlay is possible for video.
- **Production hardening.** Authentication, rate limiting, persistent encrypted storage, monitoring and a hosting plan with enough memory for Grad-CAM and longer videos.
- **Broader inputs.** Audio-visual and temporal models for video, and analysis beyond faces, are outside the scope of this work.

7.1.5 Closing Remarks

Verif-AI shows how a deepfake detector can be built as an inspectable, privacy-conscious tool and, more importantly, how its claims should be tested and bounded. A detector is only as trustworthy as the evidence about where it fails, and this report has tried to give that evidence the same weight as the architecture.

Appendix A

Source Code Snippets

This appendix reproduces excerpts of the source code (unchanged) from the repository files named in each caption.

A.1 The Custom SRM Filter Layer and the Two-Stream Network

```python
class SRMLayer(nn.Module):
    """Fixed (non-trainable) high-pass residual filters. Kernels are those of the original project."""

    def __init__(self):
        super().__init__()
        k = np.array([
            [[-1, 2, -1], [2, -4, 2], [-1, 2, -1]],
            [[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]],
            [[0, -1, 0], [-1, 4, -1], [0, -1, 0]],
        ], dtype=np.float32) / 4.0
        self.kernel = nn.Parameter(torch.tensor(k).view(3, 1, 3, 3).repeat(1, 3, 1, 1), requires_grad=False)

    def forward(self, x):
        return F.conv2d(x, self.kernel, stride=1, padding=1)


class TwoStreamNet(nn.Module):
    """RGB stream + SRM-noise stream (two EfficientNet-B0), concatenated 2x1280 -> 512 -> 1 -> sigmoid.
    Attribute names match the original checkpoints, so they load with strict=True."""

    def __init__(self, pretrained: bool = False):
        super().__init__()
        make = EfficientNet.from_pretrained if pretrained else EfficientNet.from_name
        self.spatial_net = make("efficientnet-b0")
        self.srm_layer = SRMLayer()
        self.noise_net = make("efficientnet-b0")
        self.fc = nn.Sequential(nn.Linear(1280 * 2, 512), nn.ReLU(), nn.Dropout(0.5), nn.Linear(512, 1), nn.Sigmoid
      ())

    def logits(self, x):
        """Pre-sigmoid score. forward(x) == sigmoid(logits(x)); used for numerically stable training."""
        f1 = F.adaptive_avg_pool2d(self.spatial_net.extract_features(x), 1).flatten(1)
        f2 = F.adaptive_avg_pool2d(self.noise_net.extract_features(self.srm_layer(x)), 1).flatten(1)
        return self.fc[:-1](torch.cat((f1, f2), dim=1))

    def forward(self, x):
        return torch.sigmoid(self.logits(x))
```

Listing A.1: `backend/verifai/model.py`: `SRMLayer` and `TwoStreamNet`.

A.2 Preprocessing Shared by Training and Inference

```python
def jpeg_roundtrip(face_0_255: torch.Tensor, quality: int = TRAINING_JPEG_QUALITY) -> torch.Tensor:
    """(3,H,W) float 0..255 -> same shape after JPEG encode/decode exactly as training crops were stored."""
    arr = face_0_255.detach().cpu().float().clamp(0, 255).round().byte().permute(1, 2, 0).numpy()
    ok, buf = cv2.imencode(".jpg", cv2.cvtColor(arr, cv2.COLOR_RGB2BGR), [cv2.IMWRITE_JPEG_QUALITY, int(quality)])
    if not ok:
        raise RuntimeError("JPEG encoding failed")
    img = Image.open(io.BytesIO(buf.tobytes())).convert("RGB")
    return torch.from_numpy(np.array(img)).permute(2, 0, 1).float()


def normalise(x_0_255: torch.Tensor) -> torch.Tensor:
    """(N,3,H,W) float 0..255 -> ImageNet-normalised; resized to 224 if needed."""
    x = x_0_255.float() / 255.0
    if tuple(x.shape[-2:]) != (INPUT_SIZE, INPUT_SIZE):
        x = torch.nn.functional.interpolate(x, size=(INPUT_SIZE, INPUT_SIZE), mode="bilinear", align_corners=False)
```

```
    return (x - IMAGENET_MEAN) / IMAGENET_STD


def preprocess_faces(crops: Sequence[torch.Tensor], match_training_jpeg: bool = True) -> torch.Tensor:
    """List of (3,H,W) crops (0..255) -> (N,3,224,224) model input."""
    if not crops:
        return torch.empty(0, 3, INPUT_SIZE, INPUT_SIZE)
    proc: List[torch.Tensor] = [jpeg_roundtrip(c) if match_training_jpeg else c.detach().cpu().float() for c in
      crops]
    return normalise(torch.stack(proc))
```

Listing A.2: `backend/verifai/preprocess.py`: JPEG round-trip and normalisation.

A.3 Scoring Rules

```
def decisiveness(p_fake: float) -> Tuple[float, str]:
    d = float(abs(p_fake - DECISION_THRESHOLD) * 2.0)
    return d, ("low" if d < 0.3 else "moderate" if d < 0.7 else "high")


def classify(p_fake: float, margin: float) -> Classification:
    if abs(p_fake - DECISION_THRESHOLD) < margin:
        return Classification.INCONCLUSIVE
    return Classification.FAKE if p_fake > DECISION_THRESHOLD else Classification.REAL


def label_face(p_fake: float) -> Classification:
    """Per-face label without the abstention band (raw threshold)."""
    return Classification.FAKE if p_fake > DECISION_THRESHOLD else Classification.REAL
```

Listing A.3: `backend/verifai/scoring.py`: decisiveness and the INCONCLUSIVE band.

A.4 Face Detection

```
    def detect(self, rgb: np.ndarray) -> List[DetectedFace]:
        """rgb: (H,W,3) uint8. Detection runs on a downscaled copy; crops are taken from the original."""
        h, w = rgb.shape[:2]
        scale = min(1.0, self.s.detection_max_side / max(h, w))
        small = cv2.resize(rgb, (max(1, int(round(w * scale))), max(1, int(round(h * scale)))), interpolation=cv2.
      INTER_AREA) if scale < 1 else rgb
        try:
            with self._lock:
                boxes, probs = self.mtcnn.detect(small)
        except Exception:        # extremely small / degenerate images
            return []
        if boxes is None or len(boxes) == 0:
            return []
        boxes = np.asarray(boxes, dtype=np.float64) / scale
        probs = np.asarray(probs, dtype=np.float64).reshape(-1)
        keep = []
        for i, (b, p) in enumerate(zip(boxes, probs)):
            side = min(b[2] - b[0], b[3] - b[1])
            if p >= self.s.face_min_confidence and side >= self.s.face_min_px:
                keep.append(i)
        if not keep:
            return []
        keep.sort(key=lambda i: -((boxes[i][2] - boxes[i][0]) * (boxes[i][3] - boxes[i][1])))
        keep = keep[: self.s.max_faces_per_image]
        sel = boxes[keep]
        with self._lock:
            crops = self.mtcnn.extract(rgb, sel, None)
        if crops.ndim == 3:
            crops = crops.unsqueeze(0)
        out = []
        for j, i in enumerate(keep):
            b = boxes[i]
            side = float(min(b[2] - b[0], b[3] - b[1]))
            out.append(DetectedFace(box=[float(v) for v in b], confidence=float(probs[i]), crop=crops[j],
                                    min_side_px=side, low_resolution=side < self.s.face_warn_px))
        return out
```

Listing A.4: `backend/verifai/faces.py`: detection on a downscaled copy with confidence and size filters.

A.5 Grad-CAM

```
    def heatmaps(self, batch: torch.Tensor) -> Optional[Sequence[np.ndarray]]:
        """(N,3,224,224) normalised batch -> list of (224,224,3) uint8 overlays, or None if unavailable."""
        if not self.available or batch.shape[0] == 0:
            return None
        from pytorch_grad_cam import GradCAM
        from pytorch_grad_cam.utils.image import show_cam_on_image
        from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

        base = denormalise_to_uint8(batch).astype(np.float32) / 255.0
        out = []
        with self.bundle.lock:
            m = self.bundle.model
            m.eval()
            with torch.enable_grad():
                cam = GradCAM(model=m, target_layers=[m.spatial_net._blocks[-1]])
                try:
                    gray = cam(input_tensor=batch.to(self.bundle.device), targets=[ClassifierOutputTarget(0)] *
  batch.shape[0])
                finally:
                    try:
                        cam.activations_and_grads.release()
                    except Exception:
                        pass
        for i in range(batch.shape[0]):
            out.append(show_cam_on_image(base[i], gray[i], use_rgb=True))
        return out
```

Listing A.5: `backend/verifai/explain.py`: Grad-CAM on the last block of the RGB stream.

Appendix B

API and Configuration Reference

The endpoints are listed in Table 4.1. The main configuration variables (read from the environment or a `.env` file) are:

Table B.1: Backend environment variables.

Variable	**Meaning**
`VERIFAI_MODEL_PATH`	Default checkpoint (if unset: the first existing of E4, Model 4, v2, original prototype)
`VERIFAI_ENABLE_MODEL_SELECTION`	`true`: other registered models can be chosen per analysis; `false` on hosted instances
`VERIFAI_STORAGE_DIR`	Folder for results and evidence images
`VERIFAI_CORS_ORIGINS`	Comma-separated browser origins allowed to call the API
`VERIFAI_PUBLIC_HISTORY`	`false` makes the global list of analyses empty
`VERIFAI_RESULT_TTL_HOURS`	Retention time of results (default 72)
`VERIFAI_MAX_IMAGE_MB`, `VERIFAI_MAX_VIDEO_MB`	Upload limits (defaults 15 and 200)
`VERIFAI_ENABLE_HEATMAPS`	Switch Grad-CAM on or off
`VERIFAI_INFER_BATCH_SIZE`, `VERIFAI_VIDEO_BATCH_SIZE`	Batch sizes (memory knobs; scores do not depend on them)
`VERIFAI_DEVICE`	`auto`, `cpu` or a CUDA device
`VITE_API_BASE_URL` (frontend)	Public origin of the hosted API, set at build time

Appendix C

Dataset and Licence Review

Candidate training datasets were reviewed from their descriptions and by profiling their files (counts, formats, resolution, file size, sharpness; 916,062 images in 26 folders) in a private notebook that exported statistics only. Licence badges shown by the hosting site were *not* accepted as proof; the terms of the upstream sources were applied. This is an engineering review and not legal advice. The intended use is a non-commercial academic project.

Table C.1: Decisions of the dataset review.

Dataset	**Finding**	**Decision**
RealVsFake 162k	Reals are FFHQ and CelebA (non-commercial terms), contradicting the declared CC0 badge; about half of the real images are PNG while 99% of the fakes are JPG (format shortcut).	Excluded
DeepFakeFusion-399K	Composite of Celeb-DF, FaceForensics++ (access agreement) and other sources, declared MIT; overlaps the Celeb-DF test data used here.	Excluded
DeepDetect-2025	Fakes from modern generators; reals of unspecified origin; test-split reals differ strongly from train-split reals; non-face categories present.	Fake side only, face-filtered
Real vs AI Generated Faces	FFHQ reals (upstream CC BY-NC-SA 4.0) and SFHQ fakes (upstream CC0) with clear provenance; other folders of unestablished provenance; `dataset/` folder duplicates `data_source/`.	FFHQ and SFHQ only
DeepFakeFace, Celeb-DF v2	As in earlier experiments; Celeb-DF terms allow research use only.	Used

Attribution. FFHQ: T. Karras, S. Laine and T. Aila, NVIDIA, CC BY-NC-SA 4.0, photographs by many Flickr users under their own licences; SFHQ: Synthetic Faces High Quality (CC0); Celeb-DF v2: Li et al.; DeepFakeFace: Song et al.

Appendix D

Reproducibility

- **Tests.** Backend: `cd backend && python -m pytest -q`. Frontend: `cd frontend && npm test && npx tsc -noEmit && npm run build`. End-to-end: `npx playwright test` against a running server.
- **Training notebooks.** `kaggle/build_*_bundle.py` build the private notebooks; third-party data stay inside the notebook and only checkpoints and metrics are exported.
- **Result files.** The experiment records referred to in Chapter 6 are in `docs/results/` and the pre-registered protocols in `docs/` (`STAGE0_PROTOCOL.md`, `E3_PROTOCOL.md`, `E4_PROTOCOL_ADDENDUM.md`).
- **This report.** The tables of Chapter 6 are generated by `report/scripts/make_tables.py` from the saved result files.

Bibliography

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014. URL https://arxiv.org/abs/1406.2661.

[2] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4396–4405, 2019. doi: 10.1109/CVPR.2019.00453.

[3] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. FaceForensics++: Learning to detect manipulated facial images. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1–11, 2019. doi: 10.1109/ICCV.2019.00009.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL https://arxiv.org/abs/2006.11239.

[5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10674–10685, 2022. doi: 10.1109/CVPR52688.2022.01042.

[6] Luisa Verdoliva. Media forensics and DeepFakes: An overview. *IEEE Journal of Selected Topics in Signal Processing*, 14(5):910–932, 2020. doi: 10.1109/JSTSP.2020.3002101.

[7] Ruben Tolosana, Ruben Vera-Rodriguez, Julian Fierrez, Aythami Morales, and Javier Ortega-Garcia. DeepFakes and beyond: A survey of face manipulation and fake detection. *Information Fusion*, 64: 131–148, 2020. doi: 10.1016/j.inffus.2020.06.014.

[8] Sheng-Yu Wang, Oliver Wang, Richard Zhang, Andrew Owens, and Alexei A. Efros. CNN-generated images are surprisingly easy to spot... for now. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8692–8701, 2020. doi: 10.1109/CVPR42600.2020.00872.

[9] Utkarsh Ojha, Yuheng Li, and Yong Jae Lee. Towards universal fake image detectors that generalize across generative models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24480–24489, 2023. doi: 10.1109/CVPR52729.2023.02345.

[10] Riccardo Corvi, Davide Cozzolino, Giada Zingarini, Giovanni Poggi, Koki Nagano, and Luisa Verdoliva. On the detection of synthetic images generated by diffusion models. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2023. doi: 10.1109/ICASSP49357.2023.10095167.

[11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017. URL https://arxiv.org/abs/1706.04599.

[12] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.

[13] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of StyleGAN. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8107–8116, 2020. doi: 10.1109/CVPR42600.2020.00813.

[14] Darius Afchar, Vincent Nozick, Junichi Yamagishi, and Isao Echizen. MesoNet: A compact facial video forgery detection network. In *IEEE International Workshop on Information Forensics and Security (WIFS)*, pages 1–7, 2018. doi: 10.1109/WIFS.2018.8630761.

[15] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1800–1807, 2017. doi: 10.1109/CVPR.2017.195.

[16] Peng Zhou, Xintong Han, Vlad I. Morariu, and Larry S. Davis. Two-stream neural networks for tampered face detection. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1831–1839, 2017. doi: 10.1109/CVPRW.2017.229.

[17] Belhassen Bayar and Matthew C. Stamm. A deep learning approach to universal image manipulation detection using a new convolutional layer. In *ACM Workshop on Information Hiding and Multimedia Security (IH&MMSec)*, pages 5–10, 2016. doi: 10.1145/2909827.2930786.

[18] Hao Dang, Feng Liu, Joel Stehouwer, Xiaoming Liu, and Anil K. Jain. On the detection of digital face manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5780–5789, 2020. doi: 10.1109/CVPR42600.2020.00582.

[19] Yuezun Li, Xin Yang, Pu Sun, Honggang Qi, and Siwei Lyu. Celeb-DF: A large-scale challenging dataset for DeepFake forensics. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3204–3213, 2020. doi: 10.1109/CVPR42600.2020.00327.

[20] Brian Dolhansky, Joanna Bitton, Ben Pflaum, Jikuo Lu, Russ Howes, Menglin Wang, and Cristian Canton Ferrer. The DeepFake detection challenge (DFDC) dataset. arXiv:2006.07397, 2020. URL https://arxiv.org/abs/2006.07397.

[21] Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3):868–882, 2012. doi: 10.1109/TIFS.2012.2190402.

[22] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1905.11946.

[23] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. doi: 10.1109/ICCV.2017.74.

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[25] Haixu Song, Shiyu Huang, Yinpeng Dong, and Wei-Wei Tu. Robustness and generalizability of deepfake detection: A study with diffusion models (DeepFakeFace dataset). arXiv:2309.02218, 2023. URL https://arxiv.org/abs/2309.02218.

[26] Rasmus Rothe, Radu Timofte, and Luc Van Gool. Deep expectation of real and apparent age from a single image without facial landmarks. *International Journal of Computer Vision*, 126(2–4):144–157, 2018. doi: 10.1007/s11263-016-0940-3.

[27] David Beniaguev. Synthetic faces high quality (SFHQ) dataset. Public dataset (CC0), GitHub repository SelfishGene/SFHQ-dataset. URL https://github.com/SelfishGene/SFHQ-dataset.

[28] Yuchen Luo, Yong Zhang, Junchi Yan, and Wei Liu. Generalizing face forgery detection with high-frequency features. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16312–16321, 2021. doi: 10.1109/CVPR46437.2021.01605.

[29] Lingzhi Li, Jianmin Bao, Ting Zhang, Hao Yang, Dong Chen, Fang Wen, and Baining Guo. Face X-ray for more general face forgery detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5000–5009, 2020. doi: 10.1109/CVPR42600.2020.00505.

[30] David Güera and Edward J. Delp. Deepfake video detection using recurrent neural networks. In *IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, pages 1–6, 2018. doi: 10.1109/AVSS.2018.8639163.

[31] Iacopo Masi, Aditya Killekar, Royston Marian Mascarenhas, Shenoy Pratik Gurudatt, and Wael AbdAlmageed. Two-branch recurrent network for isolating deepfakes in videos. In *European Conference on Computer Vision (ECCV)*, Lecture Notes in Computer Science, pages 667–684, 2020. doi: 10.1007/978-3-030-58571-6_39.

[32] Kaipeng Zhang, Zhanpeng Zhang, Zhifeng Li, and Yu Qiao. Joint face detection and alignment using multitask cascaded convolutional networks. *IEEE Signal Processing Letters*, 23(10):1499–1503, 2016. doi: 10.1109/LSP.2016.2603342.

[33] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

[34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1912.01703.

[35] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[36] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.